DES_Studio_Build_Plan

DES Studio — Build Plan

Living document. Update after each sprint completion.
Version: 1.0 | Created: 2026-04-30 | Grounded in: Full Codebase Audit 2026-04-30
Branch audited: `claude/audit-part-1-orientation-lhK9K`

Important: What This Plan Is

This plan **builds on the existing application**. It does not rebuild from scratch.
Every feature is classified against its actual audit status before a prompt is written.
Claude Code must read the relevant existing files before touching anything.

Audit Status Key Used Throughout

Symbol	Meaning	Action
✓	Exists and works correctly	Preserve — do not rewrite
~	Exists but incomplete or incorrect	Extend or fix the existing code
✗	Absent entirely	Build new

Sprint Status Key

Symbol	Meaning
⬜	Not started
🔄	In progress
✅	Complete
⏭	Deferred
❌	Cancelled

Direction of Travel

DES Studio is moving from a working Forms/Tabs DES modeller into a richer modelling platform with three authoring modes over one canonical `model_json`: manual Forms/Tabs, AI Generated Model, and Visual Designer. The immediate priority is to put the small platform foundation in place before the next schema-heavy modelling sprint.

Roadmap Snapshot

Stage	Status	Summary
Foundations and core DES engine	✅ Complete	Engine safety, safe evaluation, seeded reproducibility, validation, editors, experiment controls, replication, results, exports, and accessibility are in place.
AI results analysis	✅ Complete	Read-only AI Insights uses a Supabase Edge Function proxy; user-facing run labels and history exports are implemented.
Platform foundation	✅ Complete	Sprint 7B implemented the accepted 7A decisions: platform role/settings persistence and TypeScript tooling/contracts.
Dynamic modelling	✅ Complete	Sprint 7 adds time-varying arrival rates and resource capacity schedules.
AI model creation	✅ Complete	Sprint 8A prepared provider-neutral LLM routing; Sprint 8 added natural-language model authoring.
Model definition coherence	✅ Complete	Sprint 8B aligned queue/customer/server/service semantics before visual designer work.
Visual authoring	🔄 Current	Sprint 9 delivered the reviewable graph-first authoring model; Sprint 9B consolidates UX hardening, delete, richer validation, and resource editing.

Key Issues and Watchpoints

Area	Status	Direction
Shift capacity mapping	✅ Resolved	Sprint 7 uses server instance scaling: increases create idle servers; decreases retire idle excess; busy excess servers finish and warn.
TypeScript adoption	✅ Decided	ADR-009 permits incremental TypeScript at schema/domain boundaries; no broad rewrite. Sprint 7B adds tooling/contracts.
Roles and settings	✅ Implemented foundation	ADR-008 defines `profiles.role` and a dedicated `user_settings` table; Sprint 7B added migration, wrappers, and tests. Remote

Area	Status	Direction
		DB migration still needs applying before settings UI depends on it.
Dependency audit	🔄 Watch	`npm audit` reports 4 moderate findings via Vite/Vitest/esbuild; available fix requires breaking Vite 8 upgrade, so defer to dependency-maintenance pass.
Time-varying arrivals refinement	⚠️ Track	Sprint 7 samples piecewise schedules by current clock and records RATE_CHANGE markers. A later refinement can consider resampling/cancelling already-pending arrivals that cross a rate boundary if stricter NHPP behaviour is required.
LLM provider coupling	✅ Preflight complete	Sprint 8A moved browser calls to a provider-neutral contract and made provider/model selection server-side in `llm-proxy`.
SaaS tenancy/workspaces	⏭ Deferred	Important, but not required before Sprints 7-9 unless product requirements change. Needs separate schema/RLS planning.
Execute running-model UX	⏭ Deferred	MVP works; redesign should be handled in a dedicated UX/refinement sprint.
Visual Designer canvas decision	✅ Resolved	ADR-010 accepts `@xyflow/react`, optional layout-only graph metadata, and derived topology.
Model action vocabulary coherence	🔄 Active	UI observations show macro-string authoring is leaking into modeller workflows. Pause further feature expansion until ARRIVE/ASSIGN/COMPLETE/routing semantics are coherent across Forms, AI, validation, and engine.

Document History

Version	Date	Change
1.0	2026-04-30	Initial plan from audit findings
1.1	2026-05-01	Post-Sprint 1 update — sprint status recorded; Sprint 1 completion gate results noted
1.2	2026-05-03	Added Sprint 6 (LLM Integration & Results Analysis), Sprint 7 (Dynamic Distributions & Time-Varying Resources), and Sprint 8 (LLM Chat Model Builder). Features derived from design sessions in chats

Version	Date	Change
		"LLM model integration and results analysis", "dynamic sampling distributions and time varying resources", and "Building models through LLM chat interface". Absent-feature register updated.
1.3	2026-05-03	Sprint 2 complete — 215 tests passing (14 files), build 3.29s. Sprint 3 started.
1.4	2026-05-03	Added F5.9 — model delete UI with owner-only guard. Identified gap: no UI surface for model deletion existed in any prior sprint.
1.5	2026-05-04	Sprint 3 complete — Experiment Controls. Includes warm-up period, termination conditions, unified experiment panel, DB layer tests, engine test completion, and ADR-002 resolution (fork model). Noted persistent 'JS heap out of memory' error during full/engine test runs.
1.6	2026-05-04	Resolved Vitest heap issue. Root cause was unbounded seeded engine tests running open-ended `runAll()` simulations with full snapshot logs. Full suite now passes: 17 files, 272 tests.
1.7	2026-05-04	Added ADR-006 for Sprint 4 replication architecture: bounded worker pool, local live progress callbacks, one persisted batch row, and cancellation.
1.8	2026-05-04	Added detailed implementation prompts and completion checklists for Sprint 4 tasks F4.1–F4.5.
1.9	2026-05-04	Added detailed implementation prompts and completion checklists for Sprint 5 tasks F5.1–F5.9.
1.10	2026-05-04	Reviewed Sprints 6–8 prompts. Aligned LLM work with Supabase Edge Function proxy, corrected Sprint 7 references to current engine structures, resolved stale ADR numbering collisions, and flagged shift capacity mapping as an open design decision.
1.11	2026-05-04	Sprint 4 complete — Replication & Results. Added Web Worker wrapper, bounded replication runner, batch persistence via results_json, live Execute CI dashboard, cancellation, and 30-replication M/M/1 CI gate. Full suite passes: 22 files, 294 tests.
1.12	2026-05-04	Sprint 5 started. F5.1 complete — shared React ErrorBoundary added and risky library/detail/editor/execute surfaces wrapped. Focused UI tests pass.
1.13	2026-05-04	F5.2 complete — ModelDetail can export current model JSON with validation warning, stable filename slug, app version, and object URL cleanup.
1.14	2026-05-04	F5.3 complete — model library imports exported or raw JSON, validates before save, blocks invalid imports, and forces imported models private/current-user owned.
1.15	2026-05-04	F5.4 complete — Execute panel exports completed results as JSON/CSV, including experiment config, replication rows, aggregate confidence intervals, and partial batch filenames for cancelled/error runs.

Version	Date	Change
1.16	2026-05-04	F5.5 complete — model cards and overview derive run counts from user-scoped simulation history, with non-blocking placeholders and warning state if stats cannot load.
1.17	2026-05-04	F5.6 complete — simulation run saves now map `summary.avgSvc` to `avg_service_time`, keep service metrics in `results_json`, and read history fallback values from JSON for older records.
1.18	2026-05-04	F5.7 complete — keyboard/ARIA pass added labelled shared fields, focusable model cards, selected tab state, dialog semantics, execute control labels, and accessible validation alerts.
1.19	2026-05-04	F5.8 complete — empty personal libraries now show compact onboarding actions for blank creation, JSON import, and a private runnable M/M/1 sample model.
1.20	2026-05-04	F5.9 complete — owner-only model delete added with confirmation, local removal, DB ownership guard on `owner_id`, and tests for missing user/id and non-owner public models.
1.21	2026-05-04	Sprint 5 complete — Polish, Export & Production. Full test suite passes: 31 files, 334 tests. Production build succeeds.
1.22	2026-05-04	ADR-007 accepted. Planning revised around three authoring modes over one canonical `model_json`: Forms/Tabs, AI Generated Model, and Visual Designer. The old split-pane SVG hybrid designer is retired; future visual designer work should target the final graph-first authoring surface directly.
1.23	2026-05-04	Sprint 6 complete — AI assistant panel, prompt builders, streaming API client, deployed Supabase `llm-proxy` Edge Function, comparison/sensitivity actions, and tests added. Full suite passes: 32 files, 343 tests. Production build succeeds.
1.24	2026-05-05	Added platform architecture planning before further feature expansion: proposed ADR-008 for roles/SaaS/LLM configuration, proposed ADR-009 for TypeScript reassessment, and Sprint 7A architecture-readiness work. Sprint 7A scope narrowed to admin roles, user settings, and TypeScript reassessment; SaaS tenancy, LLM provider switching, architecture health review, and Execute UX direction remain planned follow-ons.
1.25	2026-05-05	Sprint 7A complete — accepted ADR-008 for platform roles/user settings and ADR-009 for incremental TypeScript adoption. CLAUDE.md updated; Sprint 7 restored as current implementation sprint.
1.26	2026-05-05	Replanned post-7A sequencing. Added Sprint 7B as the immediate implementation foundation for roles, user settings, and TypeScript tooling/contracts before Sprint 7. Added Sprint 8A LLM provider architecture preflight before AI model authoring.
1.27	2026-05-05	Added front-of-document roadmap graphic and status snapshot to make sprint sequence, current direction, and key open issues explicit.

Version	Date	Change
1.28	2026-05-05	Sprint 7B complete — added platform role/settings migration, user settings DB wrappers, `isAdmin` profile exposure, TypeScript tooling, first typed contracts, and tests. Sprint 7 is now current next.
1.29	2026-05-05	Updated front roadmap/status after Sprint 7B completion. Recorded remote migration application and dependency audit as watchpoints.
1.30	2026-05-05	Sprint 7 complete — implemented piecewise time-varying distributions, RATE_CHANGE/SHIFT_CHANGE events, shift schedule editing, validation, typed schema updates, and tests. Shift capacity mapping resolved as server instance scaling. Sprint 8A is now current next.
1.31	2026-05-05	Sprint 8A complete — added provider-neutral LLM request contract, browser API compatibility, server-side provider/model routing in `llm-proxy`, and focused LLM/Execute tests. Sprint 8 is now current next.
1.32	2026-05-05	Sprint 8 implementation pass — added AI Generated Model tab, model-builder prompts, non-streaming provider-neutral model-builder calls, structured diff preview, validation-before-apply, partial section apply, and tests. Full suite passes: 42 files, 385 tests. Production build succeeds. Live proxy deployment/manual AI proposal check remains pending.
1.33	2026-05-05	Sprint 8 AI proposal save refinement — proposal panel now offers direct Apply & Save actions for all or selected sections, preserves the real model identity when saving generated drafts, and allows invalid proposals to be saved as editable drafts with validation warnings.
1.34	2026-05-05	Added Sprint 8B — Model Definition Coherence as a blocking stabilisation pass before Sprint 9. Scope covers queue/customer binding, service-start/service-complete semantics, user-facing action labels, validation parity, AI proposal normalization, and a two-stage reference model.
1.35	2026-05-05	Continued Sprint 8B observation fixes: clarified queue/entity/B-event/C-event wording, removed remaining visible follow-on implementation language, added in-panel unsaved-change save action, summarised server resources on model cards, and replaced raw modified-proposal JSON with friendly field summaries.
1.36	2026-05-05	Started Sprint 9A and accepted ADR-010 — Visual Designer will use `@xyflow/react`; `model_json.graph` is optional layout metadata only; graph topology is derived from canonical model logic; inspector reuse strategy is focused building-block reuse.
1.37	2026-05-05	Started Sprint 9 implementation with dependency-free graph derivation helpers. Added canonical model → visual graph mapping for Source, Queue, Activity, Sink, preserving optional layout metadata and deriving edges from ARRIVE/ASSIGN/RELEASE/COMPLETE logic.
1.38	2026-05-05	Added the first Visual Designer shell in `ModelDetail`: a new tab renders derived graph counts, nodes, and connections from canonical

Version	Date	Change
		`model_json`; exports now preserve optional `model_json.graph` layout metadata.
1.39	2026-05-05	Installed `@xyflow/react` and added the first read-only Visual Designer canvas renderer with Source, Queue, Activity, and Sink node styling over the derived canonical graph.
1.40	2026-05-05	Added the initial reviewable Visual Designer authoring model: draggable layout persistence, button-based node creation, conservative canonical connection rules, compact inspector, and round-trip tests. UX polish remains the next Sprint 9 refinement pass.
1.41	2026-05-06	Continued Sprint 9 UX refinement: palette items can be dragged onto the canvas with persisted placement, node handles now follow Source/Queue/Activity/Sink port rules, and a compact visual validation summary highlights incomplete routes.
1.42	2026-05-06	Expanded the Visual Designer inspector with existing `DistPicker` controls for Source inter-arrival schedules and Activity service-time schedules, writing back to canonical B-event and C-event schedule data.
1.43	2026-05-06	Closed Sprint 9 around the reviewed initial Visual Designer authoring model and added Sprint 9B — Visual Designer UX Hardening to consolidate remaining resource editing, delete, richer validation, connection editing, and palette polish work.

Sprint History

Sprint	Status	Completed	Description	Tests Passed (Total)	M/M/1 Error	Build	Note
Pre-Sprint	✅ Complete	2026-04-30	Project setup, initial audit & plan.	~120 (120)	N/A	Success	Initia .env.
Sprint 1	✅ Complete	2026-05-03	Engine safety and correctness hardening.	182 (182)	1.48%	Success	Fixes queu valid
Sprint 2	✅ Complete	2026-05-03	UI Editor Completeness.	215 (215)	N/A	Success	Conf Conc

Sprint	Status	Completed	Description	Tests Passed (Total)	M/M/1 Error	Build	Note
Sprint 3	✅ Complete	2026-05-04	Experiment Controls (Warm-up, Termination, Fork Model).	272 (272)	1.48%	Success	ADR heap pass
Sprint 4	✅ Complete	2026-05-04	Replication & Results (Workers, Batches, CI Dashboard).	294 (294)	CI contains 9.0	Success	Bour imple insid comi
Sprint 5	✅ Complete	2026-05-04	Polish, Export & Production.	334 (334)	CI contains 9.0	Success	Erro impc servi onbc comp
Sprint 6	✅ Complete	2026-05-04	LLM Integration & Results Analysis.	343 (343)	N/A	Success	AI in Supa
Sprint 7A	✅ Complete	2026-05-05	Platform Foundation: Roles, Settings & TypeScript.	Docs only	N/A	N/A	ADR SaaS switc revie defe
Sprint 7B	✅ Complete	2026-05-05	Platform Foundation Implementation.	33 focused	N/A	Success	Role wrap tooli
Sprint 7	✅ Complete	2026-05-05	Dynamic Distributions & Time-Varying Resources.	369 (369)	N/A	Success	Piec sche RAT B-ev and
Sprint 8A	✅ Complete	2026-05-05	LLM Provider Architecture Preflight.	22 focused	N/A	Success	Prov brow prov depl
Sprint 8B	✅ Complete	2026-05-05	Model Definition Coherence.	Focused	N/A	Success	Stab sema work
Sprint 9A	✅ Complete	2026-05-05	Visual Designer	Docs	N/A	Success	ADR plan befo

Sprint	Status	Completed	Description	Tests Passed (Total)	M/M/1 Error	Build	Note
			Architecture Preflight.				
Sprint 9	✅ Complete	2026-05-06	Visual Designer Authoring.	38 focused	N/A	Success	Revi mod plac sum insp
Sprint 9B	🔄 In progress	—	Visual Designer UX Hardening.	Focused	N/A	Pending	Cons desi safe conr affor Revi com

---

Forward Product Roadmap

ADR-007 establishes DES Studio's model-authoring architecture: one canonical `model_json`, three authoring modes.

Sprint	Product focus	Authoring mode impact
Sprint 6	LLM Integration & Results Analysis	Adds read-only AI interpretation of completed runs; does not modify models
Sprint 7A	Platform Foundation: Roles, Settings & TypeScript	Decides admin/user role foundations, durable user settings, and TypeScript posture before further expansion
Sprint 7B	Platform Foundation Implementation	Implements the minimal role/settings/TypeScript foundation from Sprint 7A before further schema-heavy modelling work
Sprint 7	Dynamic Distributions & Time-Varying Resources	Extends the canonical model schema used by all authoring modes
Sprint 8A	LLM Provider Architecture Preflight	Introduces provider-neutral server-side LLM routing before AI model authoring
Sprint 8	AI Generated Model Authoring	Adds the second authoring mode: natural-language model proposals validated before apply
Sprint 8B	Model Definition Coherence	Aligns Forms/Tabs, AI generation, validation, and engine semantics for queues, service starts, completions, and routing

Sprint	Product focus	Authoring mode impact
Sprint 9	Visual Designer Authoring	Adds the third authoring mode: graph-first canvas editing over the same canonical model
Sprint 9B	Visual Designer UX Hardening	Polishes the third authoring mode after the core round-trip model is proven

The existing Forms/Tabs editor remains the stable manual authoring mode throughout. The retired split-pane SVG hybrid designer is not part of the forward roadmap.

What Already Exists — Audit Summary

Before reading the sprint plan, understand what the existing app provides.
Claude Code must never rewrite a working component — only extend or fix it.

Working and Must Be Preserved (✓)

Component	File(s)	Notes
Three-Phase engine — Phase A, B, C	`src/engine/index.js`	Structurally correct
All five macros	`src/engine/macros.js`	ARRIVE, ASSIGN, COMPLETE, RELEASE, RENEGE correct
~120 engine unit tests	`tests/`	Node environment, Vitest
FIFO queue discipline	`src/engine/entities.js:86–88`	Works correctly
IDLE/BUSY resource tracking	`src/engine/macros.js`	idleOf/busyOf helpers correct
EntityTypeEditor + AttrEditor	`src/ui/editors/index.jsx`	Create, patch, delete working
StateVarEditor	`src/ui/editors/index.jsx`	Working
BEventEditor with schedule rows	`src/ui/editors/index.jsx`	Working
CEventEditor + ConditionBuilder	`src/ui/editors/index.jsx`	Working structure
QueueEditor	`src/ui/editors/index.jsx`	Working (discipline bug in engine, not here)
Node CRUD (add/update/delete)	All editors	Working across all five types
Run history table	`src/ui/execute/index.jsx`	Last 20 runs displayed

Component	File(s)	Notes
VisualView	`src/ui/execute/index.jsx`	Server bays, queue lanes, entity tokens
StepLog	`src/ui/execute/index.jsx`	Phase-tagged event log
EntityTable	`src/ui/execute/index.jsx`	Entity status table
saveStatus banner	`src/ui/execute/index.jsx`	saving/saved/error states
Supabase auth + session	`src/App.jsx` , `src/db/supabase.js`	Auth listener working
Model CRUD wrappers	`src/db/models.js`	Working
user_id on models	`src/db/models.js`	Multi-user isolation in place

Partial — Needs Extending or Fixing (~)

Component	File(s)	Problem	Sprint
C-scan restart rule	`engine/index.js:121–142`	Pass-granularity, not event-granularity	S1
ConditionBuilder AND/OR	`editors/index.jsx`	Flat chains only, no nesting	S2
Operator filtering	`editors/index.jsx`	All operators shown regardless of valueType	S2
DropField Custom...	`editors/index.jsx:144–149`	Exists but feeds new Function() — must be removed	S1
Stats panel	`execute/index.jsx`	Runs count always 0, queues count omitted	S2
M/D/1 benchmark	`tests/`	Exists but uses Fixed dist, not Exponential	S1

Absent — Build New (✗)

Feature	Sprint
Seeded RNG	S1
LIFO and Priority queue discipline in engine	S1
Pre-run model validation	S1
Phase C truncation warning	S1
DistPicker (latent crash — import missing)	S1
UI test environment (jsdom)	S2
valueType operator filtering in ConditionBuilder	S2

Feature	Sprint
Per-C-event explicit priority field	S2
Drag-to-reorder C-events	S2
Attribute-based entity filter	S2
Undo/redo	S2
Unsaved-change warning	S2
CSV import for distributions	S2
Warm-up period (UI and engine)	S3
Time-based termination (engine reads max_simulation_time)	S3
Condition-based termination	S3
Experiment configuration panel	S3
DB layer tests	S3
Engine test completion	S3
Web Workers for engine	S4
Multi-replication runner	S4
Supabase real-time subscription	S4
Running mean + 95% CI	S4
React error boundaries	S5
Model export (JSON)	S5
Model import (JSON)	S5
Results export	S5
Keyboard navigation / ARIA	S5
First-run onboarding	S5
LLM-powered results analysis (natural language KPI interpretation)	S6
LLM results comparison (scenario A vs B narrative)	S6
LLM-generated sensitivity commentary	S6
AI assistant panel (inline, collapsible)	S6
Piecewise inter-arrival distribution (time-of-day rates)	S7
NHPP (Non-Homogeneous Poisson Process) arrival scheduler	S7
Resource shift schedule (capacity change B-Events)	S7
Time-varying distribution picker UI	S7
Shift schedule editor (capacity steps by time band)	S7
LLM chat model builder — intent parsing to model JSON	S8
LLM model builder — entity class suggestion from description	S8

Feature	Sprint
LLM model builder — iterative clarification loop	S8
LLM model builder — diff-preview before apply	S8
LLM model builder — validation integration	S8
Visual Designer — graph-first canvas authoring	S9
Visual Designer — node palette and connection rules	S9
Visual Designer — graph layout metadata for `model_json`	S9
Visual Designer — inspector integration with existing editor components	S9
Admin role and protected admin surface	S7A
User settings persistence	S7A
Platform role/settings persistence implementation	S7B
TypeScript tooling and first schema contracts	S7B
SaaS tenancy/workspace model	Post-7A
Provider-neutral LLM configuration	S8A
Admin-selectable LLM provider/model	S8A
TypeScript adoption decision and migration plan	S7A
Execute running-model UX redesign direction	Dedicated UX/design sprint

---

Pre-Sprint: Project Setup

Complete once before Sprint 1. These are prerequisites.

Task	Status	Notes
Place `CLAUDE.md` in project root	✅	Replaces missing/generic CLAUDE_WORKFLOW.md
Place `docs/addition1_entity_model.md`	✅	Entity schema and action vocabulary
Place `docs/decisions/ADR-001-auth-model.md`	✅	Auth rules
Place `docs/decisions/ADR-002-public-model-runs.md`	✅	Deferred decision
Create `.env.example`	✅	`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`
Confirm `npm run dev` starts	✅	—

Task	Status	Notes
Confirm `npm test` passes all ~120 engine tests	✅	Node environment

Pre-Sprint Orientation Prompt

```
Read CLAUDE.md and docs/addition1_entity_model.md in full.
Do not write any code.

This project has an existing working application. Before Sprint 1:
  1. Confirm you have read CLAUDE.md in full
  2. Confirm you have read docs/addition1_entity_model.md in full
  3. Run: npm test — report how many tests pass and how many fail
  4. Run: npm run dev — confirm it starts without errors
  5. Confirm you understand which files must NOT be rewritten:
     - src/engine/index.js (Three-Phase loop — correct structure)
     - src/engine/macros.js (five macros — correct)
     - src/ui/editors/index.jsx (existing editors — extend, don't replace)
     - src/ui/execute/index.jsx (run panel — extend, don't replace)
     - src/db/models.js (CRUD wrappers — extend, don't replace)

Report the result of each check. Flag anything that blocks Sprint 1.
```

Sprint 1 — Engine Safety & Correctness

Goal: Fix every issue that causes incorrect or misleading results.
All fixes operate on existing files — no new architecture.
Exit gate: M/M/1 benchmark exits 0. All ~120 existing tests still pass.

Status: ✅ Complete | **Started:** 2026-05-03 | **Completed:** 2026-05-03

> **Rule for this sprint:** Read the target file before proposing any change.
> Every task modifies existing code. None creates new files except test files.

Sprint 1 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 1 of DES Studio.

The existing app has a working Three-Phase engine, five correct macros,
and form-based editors for all five model element types.
Sprint 1 fixes correctness and security issues in the existing code.
It does NOT build new features.
```

```
Audit findings to fix:
  C1 — new Function() eval in conditions.js / macros.js (XSS on public
models)
  C3 — C-scan restart rule is pass-granularity (engine/index.js:121-142)
  C2 — LIFO/Priority ignored by engine (entities.js:84-88)
  C7 — Math.random() used throughout (distributions.js:84, phases.js:93,136)
  C5 — No pre-run validation (execute/index.jsx:141-147)
  C6 — Stale B-Event refs after deletion (phases.js:91,125)
  C4 — Phase C truncation silent (engine/index.js:121)
  C10 — Distribution inputs missing type="number"
(editors/index.jsx:171,241,731)
  C11 — DistPicker ReferenceError in components.jsx (missing import)

Each fix modifies existing files. No file is deleted or replaced.
The ~120 existing engine tests must still pass after every task.

Task order (strict):
  F1.1 — Remove new Function() from conditions.js, macros.js,
editors/index.jsx
  F1.2 — Fix C-scan restart rule in engine/index.js
  F1.3 — Implement LIFO/Priority in entities.js waitingOf()
  F1.4 — Add seeded RNG to distributions.js, thread through buildEngine()
  F1.5 — Add validateModel() before buildEngine(); fix C6, C10
  F1.6 — Surface Phase C truncation; fix DistPicker import (C11)
  F1.7 — Fix M/M/1 benchmark (currently M/D/1); confirm exits 0

Flag risks and dependencies before we start.
```

F1.1 — Remove new Function() Eval

Audit status: ~ (exists — must be removed and replaced)
Existing code: `conditions.js:76`, `macros.js:220`, `editors/index.jsx:144–149`
Action: Fix existing files — do not create new files except test file
Status: ✅ Complete | **Completed:** 2026-05-03

```
Re-read CLAUDE.md. Task F1.1 of Sprint 1.

Read these existing files before writing a single line:
  - src/engine/conditions.js  (find the new Function() call at line 76)
  - src/engine/macros.js      (find the new Function() call at line 220)
  - src/ui/editors/index.jsx  (find Custom... and DropField at lines 144-
149)

Show me the exact current code at each location.
Explain what the code does today and what the safe replacement must do.
```

```
The replacement evaluator goes into conditions.js (the existing file).
It must handle the predicate JSON structure from
docs/addition1_entity_model.md Section 4:
  - Resolve Entity.attr, Resource.id.status, Queue.id.length from state
object
  - Apply operators: == != < > <= >= (number); == != (string/boolean)
  - Handle AND/OR compound predicates

In editors/index.jsx:
  - Remove the Custom... option from DropField entirely
  - Fix DropField:131 so unknown stored values do not auto-activate free-
text

Write tests/engine/conditions.test.js (new file) BEFORE implementing:
  - Include a security test: predicate with variable 'process.exit' must not
execute
  - Run the test — confirm it fails on current code
  Then implement the fix in the existing files.
  Run the test — confirm it passes.
```

Completion checklist:

- ~~`grep -r "new Function" src/` returns nothing~~
- ~~`grep -r "eval(" src/` returns nothing~~
- ~~`Custom...` option absent from `DropField`~~
- ~~`DropField:131` auto-activate fixed~~
- ~~`tests/engine/conditions.test.js` passes including security test~~
- ~~All ~120 existing engine tests still pass~~

F1.2 — Fix C-Scan Restart Rule

Audit status: ~ (Phase C exists — restart rule is wrong granularity)
Existing code: `engine/index.js:121–142` — outer while loop correct, inner for-loop missing break
Action: Add `break` to existing for-loop — one line change, one new test
Status: ✅ Complete | **Completed:** 2026-05-03

```
Re-read CLAUDE.md Section 4. Task F1.2 of Sprint 1.

Read src/engine/index.js lines 115–150.
Show me the exact current Phase C loop code.

The audit confirmed: the outer while(cFired) loop is correct.
The problem is inside the for-loop — when a C-Event fires, the
```

```
scan continues to the next C-Event in the same pass rather than
restarting from Priority 1.

The fix is a single break statement inside the for-loop
after fireCEvent() is called.

Write the test in tests/engine/three-phase.test.js first:
  test('C-scan restarts from Priority 1 on any C-Event fire')
  Setup: two C-Events
    Priority 1: condition false initially, becomes true after Priority 2
fires
    Priority 2: condition true initially
  Expected: Priority 1 fires on the restart pass after Priority 2 fires
  Current (wrong): Priority 1 is skipped in the same cycle

Confirm the test FAILS on current code.
Add the break statement.
Confirm the test PASSES.
Confirm all ~120 existing tests still pass.
```

Completion checklist:

- ~~`break` added inside for loop after `fireCEvent()`~~
- ~~New test fails on pre-fix code (document the output)~~
- ~~New test passes after fix~~
- ~~All existing tests still pass~~

F1.3 — Implement Queue Discipline in Engine

Audit status: ✗ (LIFO/Priority exist in UI — engine ignores q.discipline entirely)
Existing code: `entities.js:84–88` — `waitingOf()` sorts by arrivalTime only
Action: Extend `waitingOf()` in existing `entities.js`
Status: ✅ Complete | **Completed:** 2026-05-03

```
Re-read CLAUDE.md Section 6. Task F1.3 of Sprint 1.

Read src/engine/entities.js — show me the current waitingOf() function.

Confirm: the audit found q.discipline is never read here.
The UI already stores FIFO/LIFO/PRIORITY correctly in the model.
The engine just never reads it.

Extend waitingOf() to read q.discipline and sort accordingly:
  FIFO:     sort ascending by arrivalTime (already done — keep it)
  LIFO:     sort descending by arrivalTime
```

```
  PRIORITY: sort ascending by Entity.priority attribute value;
            FIFO (arrivalTime ascending) as tiebreaker

Add tests to tests/engine/entities.test.js:
  - FIFO: entity with smallest arrivalTime selected
  - LIFO: entity with largest arrivalTime selected
  - PRIORITY: entity with smallest priority value selected
  - PRIORITY tiebreaker: equal priority → smallest arrivalTime
  - Entity filter: non-matching entities excluded before queue rule

The QueueEditor already shows LIFO and PRIORITY options correctly.
No UI changes needed — this is an engine-only fix.
Confirm all existing tests still pass.
```

Completion checklist:

- ~~`waitingOf()` reads `q.discipline`~~
- ~~LIFO and PRIORITY sort logic implemented~~
- ~~FIFO tiebreaker on equal PRIORITY values~~
- ~~Five new tests pass~~
- ~~All existing tests still pass~~

F1.4 — Add Seeded RNG

Audit status: ✗ (acknowledged gap — test.todo exists in engine tests)
Existing code: `distributions.js:84`, `phases.js:93,136` — Math.random() throughout
Action: Add mulberry32 to `distributions.js`; thread seed through `buildEngine()`
Status: ✅ | **Completed:** 2026-05-03

```
Re-read CLAUDE.md Section 9.2. Task F1.4 of Sprint 1.

Run: grep -rn "Math.random" src/engine/
List every file and line returned. These all need replacing.

Read the current buildEngine() signature in src/engine/index.js.

Changes to make in existing files:
  1. Add mulberry32() function to distributions.js (do not replace the file)
  2. Update buildEngine() signature: buildEngine(model, seed, maxCycles =
500)
  3. Create seeded rng inside buildEngine and pass to all samplers
  4. Replace every Math.random() call in engine files with the seeded rng

The mulberry32 implementation:
  function mulberry32(seed) {
```

```
    return function() {
      seed |= 0; seed = seed + 0x6D2B79F5 | 0;
      var t = Math.imul(seed ^ seed >>> 15, 1 | seed);
      t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t;
      return ((t ^ t >>> 14) >>> 0) / 4294967296;
    }
  }

Add to tests/engine/distributions.test.js:
  - Same seed produces identical sample sequence of 100 values
  - Different seeds produce different sequences

In the existing Execute panel (execute/index.jsx):
  - Add seed input field (integer, generates random default if empty)
  - Pass seed into buildEngine() call
  - Persist seed to runs table (column already exists in schema)

Confirm: grep -rn "Math.random" src/engine/ returns nothing after fix.
Confirm all existing tests still pass.
```

Completion checklist:

- ~~`mulberry32` added to `distributions.js`~~
- ~~`buildEngine(model, seed, maxCycles)` signature updated~~
- ~~`grep -rn "Math.random" src/engine/` returns nothing~~
- ~~Two reproducibility tests pass~~
- ~~Seed field in Execute panel (extends existing panel)~~
- ~~Seed persisted to `runs` table~~

F1.5 — Pre-Run Model Validation

Audit status: ✗ (no validation exists — buildEngine called directly)
Existing code: `execute/index.jsx:141–147` — Run button calls buildEngine() directly
Action: Add `validateModel()` before `buildEngine()` call in existing execute panel
Status: ✅ | **Completed:** 2026-05-03

```
Re-read CLAUDE.md Section 8 and docs/addition1_entity_model.md Section 7.
Task F1.5 of Sprint 1.

Read src/ui/execute/index.jsx lines 135–155.
Show me the exact current Run button handler and buildEngine() call.

Read src/ui/editors/index.jsx lines 171, 241, 731.
Show me the distribution parameter inputs at these lines.
Confirm they are missing type="number".
```

```
Build validateModel(model) — new function, can live in a new
src/engine/validation.js file or added to existing index.js.
Implement V1–V10 from docs/addition1_entity_model.md Section 7.

Integration in execute/index.jsx (extend existing file):
  - Call validateModel(model) before buildEngine()
  - If blocking errors: disable Run, surface errors inline
  - Errors must appear in the relevant editor tab, not only in execute panel
  - Use the existing saveStatus banner pattern for error display

Also fix in existing editors/index.jsx:
  - Add type="number" to distribution parameter inputs at lines 171, 241,
731
  - Add deletion reference guard for B-Events (C6):
    when a B-Event is deleted, check if any C-Event schedule references it
    and warn the modeller before completing the deletion

Confirm all existing tests still pass after these changes.
```

Completion checklist:

- ~~`validateModel()` implements V1–V10~~
- ~~V11 warning implemented~~
- ~~Run button disabled on validation failure~~
- ~~Errors shown in editor tabs (not execute panel only)~~
- ~~`type="number"` added at lines 171, 241, 731~~
- ~~B-Event deletion guard warns on dangling references~~

F1.6 — Surface Phase C Truncation + Fix DistPicker

Audit status: ✗ (C4) + ✗ (C11)
Existing code: `engine/index.js:121` (cap exists, silent), `shared/components.jsx` (DistPicker crashes)
Action: Extend existing engine and fix existing component
Status: ✅ | **Completed:** 2026-05-03

```
Task F1.6 of Sprint 1. Two fixes in existing files.

FIX 1 — Phase C truncation (C4):
Read src/engine/index.js around line 121 — find the existing 100-pass cap.
Show me the current code.
  - Raise cap from 100 to 500
  - When cap is hit, add a log entry AND add a warning to the
    results object returned by buildEngine()
```

```
  - In the existing Execute panel (execute/index.jsx), display an
    amber banner when the results object contains this warning

FIX 2 — DistPicker crash (C11):
Read src/ui/shared/components.jsx — find DistPicker.
Show me the current import section and the reference to DISTRIBUTIONS.
  - Add the missing import so DistPicker renders without crashing
  - Confirm the distribution type list is sourced from the registry
    (CLAUDE.md Section 9.2) not a hardcoded array

After both fixes:
  npm run dev — confirm no console errors on load
  Open a model — open the distribution picker — confirm no crash
  Confirm all existing tests still pass.
```

Completion checklist:

- ✅ ~~Phase C cap raised to 500 in existing `engine/index.js`~~
- ✅ ~~Warning returned in results object when cap hit~~
- ✅ ~~Amber banner shown in existing Execute panel~~
- ✅ ~~DistPicker missing import fixed in `components.jsx`~~
- ✅ ~~DistPicker renders without crashing~~
- ✅ ~~All existing tests still pass~~

F1.7 — Fix M/M/1 Benchmark

Audit status: ~ (benchmark exists — labelled M/M/1 but uses Fixed service dist — is actually M/D/1)
Existing code: benchmark file in `tests/` — find and fix
Action: Fix the existing benchmark — change service distribution to Exponential
Status: ✅ | **Completed:** 2026-05-03

```
Task F1.7 of Sprint 1 — Sprint exit gate.

Read the existing benchmark file in tests/.
Show me the current service distribution configuration.

The audit confirmed the service distribution is Fixed (deterministic),
making this an M/D/1 model, not M/M/1.

Fix the existing benchmark:
  - Change service distribution from Fixed to Exponential(rate=μ)
  - Parameters: λ=0.9, μ=1.0, ρ=0.9
  - Analytical mean wait = ρ / (μ × (1 − ρ)) = 9.0 time units
  - Tolerance: 5%
```

```
  - N = 10,000 arrivals
  - Use a fixed seed so the result is reproducible
  - Exit 0 if within 5% tolerance, exit 1 if not

Run: node tests/[benchmark-filename]
Report: simulated value, analytical value, % error, exit code.

Sprint 1 is not complete until this exits 0.
Record the % error here: _____
```

Completion checklist:

- ~~Benchmark uses Exponential service distribution~~
- ~~Benchmark exits 0~~
- ~~% error within 5% of 9.0 time units~~
- ~~All ~120 existing engine tests still pass~~
- ~~`npm run build` succeeds~~

Sprint 1 Completion Gate

```
npm test                              # All ~120 existing tests + new tests
pass
npm test -- three-phase               # Restart rule test passes
npm test -- distributions             # Seeded RNG reproducibility passes
npm test -- conditions                # Safe evaluator + security test
passes
npm test -- entities                  # Queue discipline tests pass
node tests/[benchmark-filename]       # Exits 0 — record % error
grep -r "new Function" src/           # Must return nothing
grep -rn "Math.random" src/engine/    # Must return nothing
npm run build                         # Succeeds
```

Sprint 1 Retrospective Prompt *(run in claude.ai)*

```
Sprint 1 of DES Studio is complete.

What was fixed: [paste completed feature list with file locations]
M/M/1 benchmark result: [paste output — simulated value, analytical, %
error]
Existing tests: [confirm count still ~120+new]
Any issues encountered: [paste drift corrections, unexpected findings]
Current CLAUDE.md: [paste]

Questions:
  1. Were any decisions made about the existing code that need an ADR?
```

```
  2. Did any existing tests break and need updating (legitimate change)?
  3. What gaps in the existing code did Sprint 1 uncover beyond the audit?
  4. What should Sprint 2 prioritise?
```

Sprint 2 — UI Editor Completeness

Goal: Extend the existing editor components to be fully correct and complete.
All existing editors (EntityTypeEditor, BEventEditor, CEventEditor, QueueEditor, ConditionBuilder)
remain in place — this sprint extends them.

Status: ✅ Complete | **Started:** 2026-05-03 | **Completed:** 2026-05-03
Prerequisite: Sprint 1 exit gate passed.

Sprint 2 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 2 of DES Studio.

The existing app has working editors for all five model element types.
Sprint 2 extends these existing components — it does not replace them.

Completed in Sprint 1: [paste]

Features (all extend existing files unless stated):
  F2.1 — Configure jsdom test environment (new: vite.config.js +
tests/setup.js)
  F2.2 — Extend ConditionBuilder: operator filtering by valueType
  F2.3 — Extend ConditionBuilder: compound AND/OR with explicit precedence
  F2.4 — Extend CEventEditor + engine: attribute-based entity filter
  F2.5 — Extend CEventEditor: explicit priority field + drag-to-reorder
  F2.6 — Fix ConditionBuilder token list stale after prop change (C8)
  F2.7 — Add undo/redo history to model editor
  F2.8 — Add unsaved-change warning to navigation
  F2.9 — Extend DistPicker: CSV import flow
  F2.10 — Fix ModelCard owner (null) and version tag (hardcoded v6)

Definition of done:
  - npm test -- ui runs (new environment) with no failures
  - Predicate Builder type-safety confirmed by test
  - npm run build succeeds
```

F2.1 — Configure jsdom UI Test Environment

Audit status: ✗ (vite.config.js set to node only — UI tests would fail)
Existing code: `vite.config.js` — `environment: 'node'` — needs `environmentMatchGlobs`
Status: ✅ | **Completed:** 2026-05-04

```
Re-read CLAUDE.md Section 12.3. Task F2.1 of Sprint 2.

Read the current vite.config.js. Show me the test configuration.

The audit confirmed: environment is 'node' only.
UI tests cannot run in this environment.

Extend vite.config.js (do not replace):
  Add environmentMatchGlobs:
    ['tests/ui/**', 'jsdom']   ← UI tests get jsdom
  Keep existing 'node' as default for engine tests.

Install required packages:
  npm install -D @testing-library/react @testing-library/user-event
                 @testing-library/jest-dom jsdom

Create tests/setup.js (new file):
  - Import @testing-library/jest-dom
  - afterEach(cleanup) from @testing-library/react
  - Mock the Supabase client (CLAUDE.md Section 12.3 mock structure)
    Tests must NEVER hit the real database.

Create tests/ui/smoke.test.jsx (new file):
  - Render a simple React element, assert it appears in document
  - Confirms jsdom environment is working

Run: npm test -- ui
Confirm smoke test passes.
Confirm existing engine tests still pass (node env unchanged).
```

Completion checklist:

- [x] ~~`vite.config.js` has `environmentMatchGlobs` (not replaced)~~
- [x] ~~Packages installed in `devDependencies`~~
- [x] ~~`tests/setup.js` with Supabase mock created~~
- [x] ~~Smoke test passes~~
- [x] ~~Existing engine tests unaffected~~

F2.2 — Extend ConditionBuilder: Operator Filtering by valueType

Audit status: ~ (operators shown but not filtered — all tokens treated as numeric)
Existing code: `editors/index.jsx` (ConditionBuilder) — extend the existing component
Status: ✅ | **Completed:** 2026-05-04

```
Re-read CLAUDE.md Section 7.5 and docs/addition1_entity_model.md Section
2.3.
Task F2.2 of Sprint 2.

Read src/ui/editors/index.jsx — find ConditionBuilder.
Show me the current operator dropdown code.
Show me where valueType is defined on tokens but not consumed (audit note).

Extend the existing ConditionBuilder (do not replace it):
  - When a variable is selected, read its valueType
  - Filter the operator dropdown based on valueType:
      number  → == != < > <= >=
      string  → == !=
      boolean → == !=
  - Change the value input widget based on valueType:
      number  → input type="number"
      string with allowedValues → select dropdown
      string without → input type="text"
      boolean → select with true/false options

Write tests/ui/shared/predicate-builder.test.jsx (new file):
  - Selecting a number variable shows 6 operators only
  - Selecting a string variable shows 2 operators only
  - Selecting a boolean variable shows 2 operators only
  - Value input is type="number" for number variables
  - Value input is dropdown when allowedValues is set
  - Type-mismatched predicate cannot be constructed via the UI

Use @testing-library/react. Query by role — never by CSS class.
```

Completion checklist:

- [x] ~~Operator dropdown filtered by `valueType` in existing ConditionBuilder~~
- [x] ~~Value input widget changes by `valueType`~~
- [x] ~~Type-mismatched predicate impossible to construct~~
- [x] ~~Six tests pass~~

F2.3 — Extend ConditionBuilder: Compound AND/OR

Audit status: ~ (flat AND/OR chains exist — no nesting, precedence undefined to user)
Existing code: `editors/index.jsx` (ConditionBuilder) — extend
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.3 of Sprint 2.

Read the existing ConditionBuilder AND/OR implementation.
Show me the current flat chain code.

Extend (do not replace) to support compound predicates matching
docs/addition1_entity_model.md Section 4.2:
  {
    "operator": "AND",
    "clauses": [
      { "variable": "...", "operator": "==", "value": "..." },
      { "variable": "...", "operator": ">=", "value": 1 }
    ]
  }

UI requirements:
  - Modeller can add AND or OR connector between predicates
  - When AND and OR are mixed at the same level, display the
    evaluation order explicitly (label each connector)
  - Serialised output must match the JSON structure above

Extend tests/ui/shared/predicate-builder.test.jsx:
  - AND compound serialises to correct nested JSON
  - OR compound serialises to correct nested JSON
  - Mixed AND/OR shows explicit evaluation order label in UI
```

Completion checklist:

- ✅ ~~AND/OR compound predicates serialise to nested JSON~~
- ✅ ~~Mixed AND/OR displays evaluation order explicitly~~
- ✅ ~~Three new tests pass~~

F2.4 — Entity Filter in C-Event (Engine + UI)

Audit status: ✗ (type-level selection only — no attribute-level filter)
Existing code: `editors/index.jsx` (CEventEditor), `engine/entities.js` (waitingOf)
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.4 of Sprint 2.

Read src/ui/editors/index.jsx — find CEventEditor and the SEIZE action
```

```
config.
Read src/engine/entities.js — find waitingOf() (extended in F1.3).

Add an optional Entity Filter section to the existing CEventEditor:
  - Uses the Predicate Builder but restricted to Entity.* variables only
  - Resource.* and Queue.* variables must not appear here
  - Serialises to: { "entityFilter": {...}, "queueRule": "FIFO" }

Extend waitingOf() in entities.js to apply entityFilter before sorting:
  1. Filter candidates to those where entityFilter is true
  2. If zero candidates pass filter, SEIZE does not fire this pass
  3. Apply queue discipline (FIFO/LIFO/PRIORITY) to filtered set

Engine unit test (add to existing entities.test.js):
  - Entity filter excludes non-matching entities before queue rule

UI test (add to predicate-builder.test.jsx):
  - Entity filter picker only shows Entity.* variables
```

Completion checklist:

- ~~Entity filter section in existing CEventEditor~~
- ~~Only `Entity.*` variables available in entity filter~~
- ~~`waitingOf()` applies filter before discipline sort~~
- ~~SEIZE does not fire when no entities pass filter~~
- ~~Engine unit test passes~~
- ~~UI test passes~~

F2.5 — C-Event Priority Field and Drag-to-Reorder

Audit status: ✗ (implicit array order — undiscoverable, no drag-to-reorder)
Existing code: `editors/index.jsx` (CEventEditor) — extend
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.5 of Sprint 2.

Read the existing CEventEditor — show me how C-Events are currently
listed and how their order is determined.

Extend CEventEditor (do not replace):
  - Display an explicit integer priority badge on each C-Event row
    (Priority 1 = highest, shown as a numbered badge)
  - Add drag-to-reorder using HTML5 drag-and-drop
    (do not add a new library without flagging first)
  - When order changes, update the priority integer on each C-Event
```

```
  - Store priority integers in model_json per C-Event

Write a UI test (add to tests/ui/editors/c-event-editor.test.jsx):
  - Priority badge visible on each C-Event row
  - Priority is shown as an integer (not hidden in array order)
```

Completion checklist:

- ~~Priority badge on each row in existing CEventEditor~~
- ~~Drag-to-reorder works and updates priority numbers~~
- ~~Priority stored in `model_json`~~
- ~~UI test passes~~

F2.6 — Fix ConditionBuilder Token List Staleness

Audit status: ✗ — `editors/index.jsx:495` — missing dependency in hook
Existing code: `editors/index.jsx` around line 495
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.6 of Sprint 2.

Read editors/index.jsx around line 495.
Show me the current token list construction code.

The audit found: token list goes stale after prop change.
This is likely a missing dependency in a useEffect or useMemo.

Fix the dependency array so the token list updates when:
  - A new entity class is added
  - An attribute is added or removed from an entity class
  - An entity class is deleted

Write a UI test (add to tests/ui/editors/c-event-editor.test.jsx):
  - Render CEventEditor with one entity class
  - Simulate adding a new entity class via the model state
  - Confirm new entity's attributes appear in variable picker
    without page reload
```

Completion checklist:

- ~~Token list updates on entity class change~~
- ~~Token list updates on attribute change~~
- ~~UI test passes~~

F2.7 — Undo/Redo History Stack

Audit status: ✗ (no history stack anywhere)
Existing code: `editors/index.jsx` — add history tracking to existing model state
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.7 of Sprint 2.

Read src/ui/editors/index.jsx — show me how model state is currently
managed.
Read src/db/models.js — show me the current save flow.

Before writing code, recommend one approach:
  Option A: In-memory history stack (undo reverts local state; user saves to
persist)
  Option B: Supabase version history (store model_json snapshots per change)

Explain the tradeoff for this specific codebase and recommend one.
I will confirm before you implement.

The history must track:
  - Add/delete/rename entity class or attribute
  - Add/delete/rename B-Event or C-Event or Queue
  - At least 20 undo steps

Keyboard shortcuts: Cmd/Ctrl+Z (undo), Cmd/Ctrl+Shift+Z (redo)
Visible undo/redo buttons in existing editor toolbar.
```

Completion checklist:

- [x] ~~Approach confirmed before implementation~~
- [x] ~~Undo/redo tracks minimum 20 steps~~
- [x] ~~Keyboard shortcuts working~~
- [x] ~~Toolbar buttons present in existing UI~~

F2.8 — Unsaved-Change Warning

Audit status: ✗ (Back button discards silently — known defect)
Existing code: `App.jsx`, existing editor navigation
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.8 of Sprint 2.

Read src/App.jsx — find the navigation / back button handler.
Show me the current code.
```

```
Extend the existing navigation (do not replace App.jsx structure):
  - Track whether current model has unsaved changes
  - When user navigates away (Back, browser navigation):
    show dialog: "You have unsaved changes. Leave without saving?"
  - If confirmed: navigate, discard changes
  - If cancelled: stay on page
  - Register beforeunload handler for browser tab close / refresh

Write a UI test (add to tests/ui/editors/ or execute):
  - Make a change to model
  - Simulate clicking Back
  - Confirm warning dialog appears
  - Confirm navigation is blocked until resolved
```

Completion checklist:

- ~~Warning dialog on navigation away~~
- ~~`beforeunload` handler registered~~
- ~~UI test passes~~

F2.9 — DistPicker CSV Import

Audit status: ✗ (DistPicker fixed in F1.6 — CSV import not yet present)
Existing code: `shared/components.jsx` (DistPicker — now working after F1.6)
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.9 of Sprint 2.

Read src/ui/shared/components.jsx — find the DistPicker component
(now working after F1.6 fix).

Extend DistPicker to add CSV import:
  1. Add "Import from CSV" to the distribution type dropdown
  2. On select: open file picker (accept .csv)
  3. Parse CSV in browser — no server upload
  4. Let user select which column contains the values
  5. Show summary: filename, column, rows accepted, skipped, min, max, mean
  6. If > 10% rows skipped (non-numeric): show warning before confirming
  7. On confirm: store values array in model JSON as:
     { "type": "empirical", "values": [...], "sourceFile": "...", "column":
"..." }

The CSV file is NEVER stored in Supabase. Values array only.
The engine has no knowledge of CSV files at runtime.
```

```
Write a UI test using a mock File object (add to tests/ui/shared/dist-
picker.test.jsx).
```

Completion checklist:

- [x] ~~CSV option in existing DistPicker dropdown~~
- [x] ~~Column selection shown after parse~~
- [x] ~~10% skip threshold warning shown~~
- [x] ~~Values array stored (no CSV file stored)~~
- [x] ~~UI test passes~~

F2.10 — Fix ModelCard Owner and Version Tag

Audit status: ✗ — `const owner = null` hardcoded; version tag hardcoded "v6"
Existing code: `App.jsx` (ModelCard), `ui/execute/index.jsx` (version tag)
Status: ✅ | **Completed:** 2026-05-03

```
Task F2.10 of Sprint 2. Two small fixes in existing files.

FIX 1 — ModelCard owner (App.jsx):
  Read App.jsx — find: const owner = null
  Fix to fetch the owner profile from the profiles table using
model.user_id.
  Display username and avatar_url from the profiles table.

FIX 2 — Version tag (execute/index.jsx or ModelDetail):
  Find the hardcoded "v6" string.
  Replace with: import pkg from '../../package.json'; then use pkg.version.

Both are cosmetic. No engine or logic changes. No new files.
```

Completion checklist:

- [x] ~~Owner avatar and username displayed in ModelCard~~
- [x] ~~Version tag reads from `package.json` dynamically~~

Sprint 2 Completion Gate

```
✅ npm test -- --run              # 215 tests passing, 14 files
✅ npm test -- ui                 # 30 tests passing, 5 files
✅ npm test -- predicate-builder  # 11 tests passing
✅ npm test -- c-event-editor     # 7 tests passing — duplicate empty-
```

```
string key warning, non-blocking
✅ npm test -- dist-picker        # 7 tests passing
✅ npm run build                  # 3.29s
```

Sprint 3 — Experiment Controls

Goal: Implement warm-up period, termination conditions, and unified experiment panel. All changes extend existing engine and UI files.

Status: ✅ Complete | **Started:** 2026-05-03 | **Completed:** 2026-05-04
Prerequisite: Sprint 2 exit gate passed.

> **Key audit finding:** `max_simulation_time` column already exists in the DB schema. The engine never reads it. Time-based termination is therefore a two-part fix: read the existing column in the engine, and add UI controls.

Sprint 3 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 3 of DES Studio.

The existing app has a working single-run engine.
Sprint 3 adds experiment controls to the existing engine and UI.

Completed in Sprints 1–2: [paste]

Features:
  F3.1 — Warm-up period: new parameter; statistics reset in ENGINE at
T_warmup
  F3.2 — Time-based termination: engine reads existing max_simulation_time
column
  F3.3 — Condition-based termination: new predicate expression evaluated per
Phase A
  F3.4 — Unified experiment configuration panel (extends existing Execute
panel)
  F3.5 — Warm-up audit trail in StepLog
  F3.6 — DB layer tests: confirm user_id isolation in existing CRUD wrappers
  F3.7 — Engine test completion: fill gaps in existing test suite
  F3.8 — ADR-002 resolution: public model run permissions decision

CRITICAL: Warm-up reset must be enforced IN THE ENGINE.
The audit found: neither UI nor engine currently implements warm-up.
A UI label that hides warm-up results while the engine aggregates them
is a correctness bug.
```

```
Definition of done:
  - Warm-up exclusion confirmed by unit test that fails before and passes
after
  - Condition termination halts simulation correctly
  - DB isolation tests confirm user_id filter on all list queries
  - npm test -- --run passes with zero failures
```

F3.1 — Warm-Up Period (Engine + UI)

Audit status: ✗ (absent from both engine and UI)
Existing code: `engine/index.js` (extend loop), `execute/index.jsx` (extend panel)
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.1 of Sprint 3.

Read src/engine/index.js — show me the full simulation loop.
Read src/ui/execute/index.jsx — show me the Run button handler and
how buildEngine() is called.

Extend buildEngine() to accept warmupPeriod parameter.
In the engine loop, at T_now >= warmupPeriod:
  - Reset all statistics collectors (running sum, count, entity sojourn
times)
  - Reset user-defined state variables where resetOnWarmup: true
  - This reset happens once only (use a flag to prevent double-reset)

Write the unit test BEFORE implementing:
  - Run with warmup period; use high service times during warmup
  - Assert mean wait time reflects only post-warmup observations
  - Confirm test FAILS on current code
  Implement the fix.
  - Confirm test PASSES

In the existing Execute panel:
  - Add warm-up period field (number input, time units)
  - Pass to buildEngine()
  - Persist to runs table (add warmup_period column — update schema.sql and
models.js)
```

Completion checklist:

- ~~`buildEngine()` accepts `warmupPeriod`~~
- ~~Stats reset in engine at `T_warmup` (not in UI)~~
- ~~Unit test fails before implementation, passes after~~
- ~~Warm-up field in existing Execute panel~~

- [x] ~~`schema.sql` and `models.js` updated with `warmup_period` column~~

F3.2 — Time-Based Termination

Audit status: ✗ (max_simulation_time column exists in DB — engine never reads it)
Existing code: `engine/index.js`, `db/models.js` (column exists), `execute/index.jsx`
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.2 of Sprint 3.

Read src/db/models.js — confirm max_simulation_time column exists in the
runs table.
Read src/engine/index.js — confirm it never reads max_simulation_time.
Read src/ui/execute/index.jsx — show me how the run is submitted.

The column already exists. The engine just never reads it.

Extend engine/index.js loop:
  - Accept maxSimTime parameter in buildEngine()
  - After Phase A clock advance: if T_now >= maxSimTime, terminate

Extend execute/index.jsx (existing Run handler):
  - Add Run Duration field (number input) to experiment panel
  - Validate: run duration must be > warmup period
  - Pass maxSimTime into buildEngine() call
  - The DB write already has the column — just populate it (currently always
null)

Unit test: engine halts at exactly T_end.
```

Completion checklist:

- [x] ~~Engine reads `maxSimTime` parameter~~
- [x] ~~Engine terminates at `T_now >= maxSimTime`~~
- [x] ~~Run Duration field in existing Execute panel~~
- [x] ~~Validation: run duration > warm-up period~~
- [x] ~~`max_simulation_time` column now populated in runs table~~
- [x] ~~Unit test passes~~

F3.3 — Condition-Based Termination

Audit status: ✗ (not implemented)
Existing code: `engine/index.js`, `execute/index.jsx`, `ConditionBuilder` (reuse)

Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.3 of Sprint 3.

Extend engine/index.js to support a termination predicate:
  - Accept terminationCondition (predicate JSON object) in buildEngine()
  - After every Phase A clock advance, evaluate the predicate
  - If true, terminate
  - If both maxSimTime and terminationCondition are set,
    whichever fires first wins

In execute/index.jsx:
  - Add Termination Condition section to experiment panel
  - Reuse the existing ConditionBuilder component
  - Same variable namespace as C-Event conditions
  - Serialise as predicate JSON and pass into buildEngine()

Unit test: engine halts when predicate becomes true (not at a fixed time).
```

Completion checklist:

- ~~`terminationCondition` parameter in `buildEngine()`~~
- ~~Engine evaluates predicate each Phase A~~
- ~~Terminates correctly when predicate true~~
- ~~UI reuses existing `ConditionBuilder` component~~
- ~~Unit test passes~~

F3.4 — Unified Experiment Configuration Panel

Audit status: ✗ (controls scattered or absent — Execute panel exists but needs extension)
Existing code: `execute/index.jsx` — extend existing panel
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.4 of Sprint 3.

Read src/ui/execute/index.jsx — show me the current panel layout.

Extend the existing Execute panel to group all experiment parameters:
  - Warm-up period (from F3.1)
  - Termination mode: radio buttons — Time-based OR Condition-based
    Time-based: shows Run Duration field
    Condition-based: shows ConditionBuilder
  - Replication count (integer, minimum 1)
    Note: multi-worker engine is Sprint 4 — this field stores the value
    but currently runs only 1 replication
```

```
  - Seed field (from F1.4)

Validation before Run:
  - Warm-up < run duration (time-based termination)
  - Replication count is a positive integer
  - Show errors inline, disable Run until resolved
```

Completion checklist:

- [x] ~~All experiment parameters in one panel section~~
- [x] ~~Termination mode radio buttons working~~
- [x] ~~Replication count field present (engine still single-run — Sprint 4)~~
- [x] ~~Validation errors shown inline before Run~~
- [x] ~~Run blocked until validation passes~~

F3.5 — Warm-Up Audit Trail in StepLog

Audit status: *X* (StepLog exists and works — add warm-up boundary marker)
Existing code: `execute/index.jsx` (StepLog — working component)
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.5 of Sprint 3.

Read the existing StepLog in execute/index.jsx.
Show me how log entries are currently rendered.

Extend (do not replace) to add a warm-up boundary marker:
  - When the engine returns results, include the T_warmup clock time
  - Insert a visual separator in the step log at that point:
    "——— Warm-up ended at T=X.XX ———"
  - In the stats panel, add: warm-up duration used, observations excluded,
    observations included

If a replication's warm-up exceeds its run duration, show an error banner.
```

Completion checklist:

- [x] ~~Warm-up boundary marker in existing StepLog~~
- [x] ~~Stats panel shows warm-up statistics~~
- [x] ~~Error banner if warm-up > run duration~~

F3.6 — DB Layer Tests

Audit status: ✗ (zero DB tests — CRUD wrappers work but are untested)
Existing code: `src/db/models.js` — tested via mocked Supabase client
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.6 of Sprint 3.

Read src/db/models.js in full — list all CRUD functions.

Write tests/db/models.test.js (new file) using the mocked Supabase
client from tests/setup.js. Never hit the real database.

Required test cases (all from CLAUDE.md Section 12.6):
  - getModels() always calls .eq('user_id', userId)
  - getPublicModels() calls .eq('is_public', true) — no user_id filter
  - saveModel() INSERT always includes user_id in payload
  - updateModel() PATCH does not overwrite is_public silently
  - deleteModel() requires a model id — no id means no execution
  - saveRun() INSERT always includes model_id

Assert on mock call arguments — not just that a result was returned.
```

Completion checklist:

- [x] ~~`tests/db/models.test.js` created~~
- [x] ~~All six critical isolation tests pass~~
- [x] ~~`user_id` filter confirmed on all list queries~~

F3.7 — Engine Test Completion

Audit status: ~ (5 test files, ~120 tests — gaps in required coverage)
Existing code: `tests/engine/` — extend existing files
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.7 of Sprint 3.

Read all existing engine test files and list what they cover.
Cross-reference against CLAUDE.md Section 12.4 required cases.
Identify missing tests only — do not rewrite passing tests.

Priority gaps to fill (add to existing test files):
  - Phase B fires ALL events at T_now (not just first)
  - FEL remains sorted after new B-Event is inserted
  - Engine terminates correctly when FEL is empty
  - All seven distribution types produce correct theoretical means
    over 10,000 samples (within 2%)
```

```
  - ARRIVE macro creates entity with correct default attribute values
  - COMPLETE macro records sojourn time correctly
  - RENEGE B-Event is a no-op if entity already seized
  - AND compound predicate evaluates both clauses
  - OR compound predicate short-circuits correctly

Do not modify existing passing tests.
```

Completion checklist:

- ✅ ~~All required engine test cases exist~~
- ✅ ~~All tests pass~~
- ✅ ~~No existing tests broken~~

F3.8 — ADR-002 Resolution

Audit status: ✗ (deferred decision — must be made before Sprint 4 builds run history)
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F3.8 of Sprint 3.

Read docs/decisions/ADR-002-public-model-runs.md.

Present the two options clearly:
  Option A: Public models are view-only for non-owners. No run permissions.
  Option B: Non-owners can run public models. Results stored under runner's
user_id.

Key tradeoff: Option B requires the runs table to reference a model
the runner does not own, which may conflict with existing RLS policies
in Supabase.

I will decide. Once decided:
  1. Update ADR-002-public-model-runs.md with the decision and rationale
  2. Update CLAUDE.md Section 17.5 ADR Register
  3. If Option B: update models.js and schema.sql accordingly

Do not implement any run permissions until I confirm the decision.
```

Completion checklist:

- ✅ ~~ADR-002 updated with decision~~
- ✅ ~~CLAUDE.md ADR Register updated~~
- ✅ ~~Code changes implemented (if Option B)~~

Sprint 3 Completion Gate

```
npm test -- --run                          # Zero failures
npm test -- db                             # DB isolation tests pass
npm test -- engine                         # All engine test cases pass
npm test -- ui                             # All UI tests pass
node tests/[benchmark-filename]            # Still exits 0
npm run build                              # Succeeds
```

Sprint 4 — Replication & Results

Goal: Enable N replications using Web Workers with independent seeds.
Stream same-browser progress via local runner callbacks, persist final batch results to Supabase, and display live confidence intervals.

Status: ✅ Complete | **Started:** 2026-05-04 | **Completed:** 2026-05-04
Prerequisite: Sprint 3 exit gate passed. ADR-002 and ADR-006 accepted.

Feature	Audit Status	Action
F4.1 — Web Worker engine wrapper	✗ (engine blocks main thread)	New: `src/engine/worker.js`
F4.2 — Multi-replication runner	✗ (replications always 1)	New: `src/engine/replication-runner.js`
F4.3 — Batch persistence + local progress callbacks	✗ (replication progress/results not wired)	Extend: `execute/index.jsx`, `src/db/models.js`
F4.4 — Running mean + 95% CI	✗ (point estimates only)	New: `src/engine/statistics.js`
F4.5 — Live CI dashboard	✗ (static results table only)	Extend: `execute/index.jsx`

Sprint 4 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 4 of DES Studio.

The existing app runs a single replication synchronously on the main thread.
Sprint 4 adds parallel replication and live results streaming.

Completed in Sprints 1–3: [paste]

Features:
  F4.1 – Web Worker wrapper for buildEngine() (non-blocking main thread)
```

```
  F4.2 — Multi-replication runner: N replications scheduled through a
bounded worker pool
  F4.3 — Local live progress callbacks; Supabase persists final batch
results
  F4.4 — Running mean and 95% CI (t-distribution for N<30)
  F4.5 — Live CI dashboard in existing execute panel

CRITICAL:
  - Seeds must be independent per worker
  - Use a bounded worker pool, not one worker per replication
  - Same-browser live progress uses local runner callbacks
  - The existing runs table has replications column (always 1) — fix this
  - Store one run row per replication batch; put per-replication and
aggregate CI results in results_json
  - batch_id should identify the batch if the runs schema supports it or is
migrated in Sprint 4
  - Active replication batches must be cancellable
  - The existing single-run Execute flow must still work if N=1

Definition of done:
  - 30 M/M/1 replications run through the bounded worker pool — UI stays
responsive
  - 95% CI contains analytical mean wait (9.0) after 30 reps
  - CI narrows visibly on dashboard as replications complete
  - npm test -- --run passes
```

F4.1 — Web Worker Engine Wrapper

Audit status: *X* (engine runs synchronously on main thread)
Action: New `src/engine/worker.js`
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F4.1 of Sprint 4.

Read these files before writing code:
  - src/engine/index.js
  - src/ui/execute/index.jsx
  - docs/decisions/ADR-006-replication-runner-architecture.md

Create src/engine/worker.js as a thin Web Worker wrapper around
buildEngine().
Do not move engine logic into the worker file. The worker only receives a
run
request, calls buildEngine(), and posts a structured response.

Worker message contract:
```

```
  Input:
    {
      type: "RUN_REPLICATION",
      payload: {
        replicationIndex,
        model,
        seed,
        warmupPeriod,
        maxSimTime,
        terminationCondition,
        maxCycles,
        maxCPasses
      }
    }

  Success output:
    {
      type: "REPLICATION_COMPLETE",
      payload: {
        replicationIndex,
        seed,
        result
      }
    }

  Error output:
    {
      type: "REPLICATION_ERROR",
      payload: {
        replicationIndex,
        seed,
        message,
        stack
      }
    }

Rules:
  - Keep src/engine pure JavaScript. No React imports, no DOM access.
  - The worker must not import any UI or DB code.
  - The worker must not persist results.
  - The worker must catch errors and post REPLICATION_ERROR rather than
crashing silently.
  - Preserve buildEngine() argument order as it exists in
src/engine/index.js.

Testing:
  - Add a unit/integration test for the worker message contract if the
Vitest environment supports Worker.
  - If Worker is awkward in Vitest, export a pure helper from worker.js,
e.g.
```

```
    runReplicationPayload(payload), and test that helper directly.
  - Test success returns replicationIndex, seed, and result.summary.
  - Test invalid model/input returns a structured error.
```

Completion checklist:

- [x] ~~`src/engine/worker.js` created~~
- [x] ~~Worker accepts `RUN_REPLICATION`~~
- [x] ~~Worker posts `REPLICATION_COMPLETE` on success~~
- [x] ~~Worker posts `REPLICATION_ERROR` on failure~~
- [x] ~~No UI/DB imports in worker~~
- [x] ~~Worker/helper tests pass~~

F4.2 — Multi-Replication Runner

Audit status: ✗ (replications field exists but execution still effectively single-run)
Action: New `src/engine/replication-runner.js`
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F4.2 of Sprint 4.

Read these files before writing code:
  - src/engine/worker.js (from F4.1)
  - src/engine/index.js
  - src/ui/execute/index.jsx
  - docs/decisions/ADR-006-replication-runner-architecture.md

Create src/engine/replication-runner.js.

Export runReplications(options):
  options = {
    model,
    replications,
    baseSeed,
    warmupPeriod,
    maxSimTime,
    terminationCondition,
    maxCycles,
    maxCPasses,
    workerCount,
    onProgress,
    onReplicationComplete,
    onError,
    onComplete,
    onCancelled
```

```
  }

Return a controller:
  {
    cancel()
  }

Runner behaviour:
  - Use ADR-006: bounded worker pool, not one worker per replication.
  - Default workerCount should be conservative:
      min(replications, max(1, (navigator.hardwareConcurrency || 2) - 1), 4)
    but allow tests to pass workerCount explicitly.
  - Assign seeds deterministically:
      seed = baseSeed + replicationIndex
    where replicationIndex is 0-based.
  - Start up to workerCount workers.
  - As each worker completes, start the next pending replication until all
are done.
  - Preserve result order by replicationIndex in the final results array.
  - Invoke onProgress with:
      { completed, total, running, pending, cancelled }
  - Invoke onReplicationComplete after every successful replication.
  - Invoke onComplete once with all replication results if not cancelled.
  - On cancel():
      terminate active workers,
      stop scheduling pending work,
      invoke onCancelled,
      do not invoke onComplete.

Single-replication compatibility:
  - If replications === 1, the existing Execute flow may still use the
    synchronous path from execute/index.jsx.
  - The runner should still work for replications === 1, because tests may
call it.

Testing:
  - Unit test seed assignment: baseSeed 100, 3 reps => 100,101,102.
  - Unit test bounded pool: replications 30, workerCount 4 never runs more
than 4 active workers.
  - Unit test final result order is replicationIndex order even if workers
finish out of order.
  - Unit test cancellation terminates active work and prevents onComplete.
  - Use fake worker adapters if needed so tests do not depend on real
browser Worker timing.
```

Completion checklist:

- [x] ~~`src/engine/replication-runner.js`~~ ~~created~~
- [x] ~~Bounded worker pool implemented~~

- [x] ~~Deterministic independent seeds implemented~~
- [x] ~~Progress callbacks implemented~~
- [x] ~~Cancellation implemented~~
- [x] ~~Result ordering preserved~~
- [x] ~~Runner tests pass~~

F4.3 — Batch Persistence + Local Progress Callbacks

Audit status: ✗ (replication batches are not persisted as batches; live progress not wired)
Action: Extend `src/ui/execute/index.jsx`, `src/db/models.js`
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F4.3 of Sprint 4.

Read these files before writing code:
  - src/db/models.js
  - src/ui/execute/index.jsx
  - docs/decisions/ADR-002-public-model-runs.md
  - docs/decisions/ADR-006-replication-runner-architecture.md

Implement ADR-006 persistence:
  - Same-browser live progress uses local runner callbacks.
  - Supabase is used for final batch persistence.
  - Store one run row per replication batch.
  - Store per-replication results and aggregate summaries in results_json.

Extend or add DB wrapper functionality in src/db/models.js:
  - saveRun() must accept:
      {
        model_id,
        seed,
        replications,
        max_simulation_time,
        results_json,
        batch_id
      }
  - Preserve existing single-run save behaviour.
  - Do not query Supabase directly from execute/index.jsx if a wrapper can
do it.
  - If the schema does not yet support batch_id, handle it deliberately:
      Option A: include batch_id only if existing code/schema supports it
      Option B: add a migration/schema note in docs and code TODO
    Do not silently assume a missing DB column exists.

Extend execute/index.jsx:
```

```
  - Generate a stable batch_id for every multi-replication run.
    Use crypto.randomUUID() when available; fallback to a timestamp/random
string.
  - Wire replication-runner callbacks into Execute component state:
      progress
      perReplicationResults
      aggregateStats
      status: idle | running | cancelling | complete | error | cancelled
  - Persist only after successful completion.
  - Do not persist a successful final result after cancellation.
  - Preserve ADR-002 fork behaviour for non-owner public model runs.

Testing:
  - DB test: saveRun includes seed, replications, results_json, and batch_id
when supplied.
  - DB test: single-run saveRun still works with replications=1.
  - UI test: progress state updates when runner callbacks fire.
  - UI test: completed multi-replication batch calls saveRun once, not once
per replication.
  - UI test: cancelled batch does not call successful saveRun.
```

Completion checklist:

- ~~`saveRun()` accepts and persists batch replication metadata~~
- ~~One run row per batch, not one row per replication~~
- ~~`results_json` contains per-replication and aggregate data~~
- ~~Execute panel uses local progress callbacks~~
- ~~Completed batch persists once~~
- ~~Cancelled batch does not persist as successful~~
- ~~DB and UI tests pass~~

F4.4 — Running Mean + 95% CI

Audit status: ✗ (only point estimates displayed)
Action: New `src/engine/statistics.js`
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F4.4 of Sprint 4.

Create src/engine/statistics.js.

Export:
  - mean(values)
  - sampleVariance(values)
  - sampleStdDev(values)
```

```
  - tCritical95(df)
  - confidenceInterval95(values)
  - summarizeReplicationResults(results, metricPaths)

Rules:
  - Ignore null, undefined, and non-finite values when computing statistics.
  - For n = 0: return { n: 0, mean: null, lower: null, upper: null,
halfWidth: null }
  - For n = 1: mean is defined, CI bounds are null because sample variance
is unavailable.
  - For n >= 2:
      halfWidth = tCritical95(n - 1) * sampleStdDev(values) / sqrt(n)
      lower = mean - halfWidth
      upper = mean + halfWidth
  - Use a small t-critical lookup table for df 1..30.
  - For df > 30 use 1.96.
  - Do not add a statistics dependency.

summarizeReplicationResults(results, metricPaths):
  - results is an array of replication payloads from the runner.
  - metricPaths might include:
      "summary.avgWait"
      "summary.avgSvc"
      "summary.served"
      "summary.reneged"
      "summary.avgSojourn"
  - Return an object keyed by metricPath with n, mean, lower, upper,
halfWidth.

Testing:
  - mean ignores null/undefined/non-finite values.
  - sampleStdDev uses n-1 denominator.
  - confidenceInterval95 for one value has null bounds.
  - confidenceInterval95 for known values matches expected half-width within
tolerance.
  - tCritical95(1), tCritical95(10), tCritical95(30), tCritical95(31) return
expected values.
  - summarizeReplicationResults extracts nested metrics correctly.
```

Completion checklist:

- ~~`src/engine/statistics.js` created~~
- ~~Mean, sample variance, sample std dev implemented~~
- ~~95% CI implemented with t-critical lookup for df <= 30~~
- ~~Non-finite values ignored~~
- ~~Nested metric summary helper implemented~~
- ~~Statistics tests pass~~

F4.5 — Live CI Dashboard

Audit status: ✗ (Execute view has static/single-run results only)
Action: Extend `src/ui/execute/index.jsx`
Status: ✅ Complete | **Completed:** 2026-05-04

```
Task F4.5 of Sprint 4.

Read src/ui/execute/index.jsx in full before editing.
Preserve the existing Execute panel, VisualView, StepLog, EntityTable,
saveStatus
banner, run history, warm-up controls, termination controls, and single-run
flow.

Extend the Execute panel for multi-replication runs:

When replications === 1:
  - Keep the existing single-run user experience.
  - Existing VisualView, StepLog, EntityTable, and summary behaviour remain
available.

When replications > 1:
  - Run through replication-runner.js.
  - Show batch status:
      Running X/Y
      Worker pool size
      Pending count
      Cancel button
  - Show a live replication results table:
      Rep #
      Seed
      Served
      Avg wait
      Avg service
      Avg sojourn
      Status
  - Show aggregate CI cards or table for key KPIs:
      mean
      lower 95%
      upper 95%
      half-width
      n
  - Update aggregates as each replication completes.
  - CI should visibly narrow as more replications finish.
  - On Cancel:
      call controller.cancel()
      disable Cancel while cancellation is in progress
```

```
      show cancelled status
      do not save successful final run
  - On worker error:
      surface an error banner
      keep completed replication results visible
      allow user to run again

UI/UX rules:
  - Use existing tokens from src/ui/shared/tokens.js.
  - Do not put cards inside cards.
  - Keep dashboard compact and operational, not a marketing/hero layout.
  - Avoid layout shifts as rows are added.
  - Do not use hardcoded style values where tokens exist.

Testing:
  - UI test: replications=1 still renders existing single-run results.
  - UI test: replications>1 shows batch progress and Cancel button.
  - UI test: fake runner completion updates per-replication table.
  - UI test: aggregate CI appears after at least two completed replications.
  - UI test: cancel calls controller.cancel() and does not call saveRun as
successful.
  - Integration/engine test: 30 M/M/1 replications with fixed base seed
produce a CI that contains analytical mean wait 9.0.
```

Completion checklist:

- ✅ ~~Existing single-run Execute flow preserved~~
- ✅ ~~Multi-replication dashboard rendered~~
- ✅ ~~Live progress updates as replications complete~~
- ✅ ~~Running CI displayed for key KPIs~~
- ✅ ~~Cancel button works~~
- ✅ ~~Successful batch persists once~~
- ✅ ~~Error/cancel states are visible~~
- ✅ ~~UI tests pass~~
- ✅ ~~30-replication M/M/1 CI gate passes~~

Sprint 5 — Polish, Export & Production

Goal: Production-ready release. Error handling, import/export, accessibility, onboarding.

Status: ✅ Complete | **Started:** 2026-05-04 | **Completed:** 2026-05-04
Prerequisite: Sprint 4 exit gate passed.

Feature	Audit Status	Action
F5.1 — React error boundaries	✗ (no ErrorBoundary anywhere)	New: extend `shared/components.jsx`
F5.2 — Model export to JSON	✗ (Supabase-only)	New: download button in ModelDetail
F5.3 — Model import from JSON	✗ (no upload)	New: import button in model library
F5.4 — Results export	✗ (absent)	New: download from Execute panel
F5.5 — Fix stats panel runs count	~ (always 0 — model.stats never populated)	Fix: `db/models.js:127` area
F5.6 — Fix avg_service_time column (C9)	~ (mismatch)	Fix: `db/models.js:127`
F5.7 — Keyboard navigation + ARIA	✗ (not implemented)	Extend: all UI components
F5.8 — First-run onboarding	✗ (not implemented)	New: welcome panel + sample model
F5.9 — Model delete	✗ (deleteModel() exists in db/models.js — no UI surface anywhere)	New: owner-only Delete button on ModelCard + `owner_id` ownership guard

Sprint 5 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 5 of DES Studio.

Sprint 5 goal: Production-ready — error handling, export/import,
accessibility, and guided onboarding.

Completed in Sprints 1–4: [paste]

Features:
  F5.1 — React error boundaries (new — extends shared/components.jsx)
  F5.2 — Model export to JSON (new — adds button to existing ModelDetail)
  F5.3 — Model import from JSON (new — adds button to existing model
library)
  F5.4 — Results export JSON/CSV (new — adds buttons to existing Execute
panel)
  F5.5 — Stats panel fix: model.stats never populated
  F5.6 — avg_service_time column fix (C9)
  F5.7 — Keyboard navigation and ARIA labels (extend all existing
components)
  F5.8 — First-run onboarding (new — welcome panel + pre-built M/M/1 model)
  F5.9 — Model delete: owner-only Delete button on ModelCard,
```

```
          confirmation dialog, deleteModel() user_id guard,
          cascade behaviour documented

Definition of done:
  - Zero console errors in any workflow path
  - Keyboard-only user can complete a full simulation run
  - Exported JSON re-imports without validation errors
  - Lighthouse accessibility score >= 85
  - npm test -- --run passes with zero failures
  - npm run build succeeds
```

F5.1 — React Error Boundaries

Audit status: ✗ (no ErrorBoundary anywhere)
Action: New shared component, wrap risky UI surfaces
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.1 of Sprint 5.

Read these files before writing code:
  - src/ui/shared/components.jsx
  - src/App.jsx
  - src/ui/ModelDetail.jsx
  - src/ui/editors/index.jsx
  - src/ui/execute/index.jsx

Add a reusable ErrorBoundary component to src/ui/shared/components.jsx.
React error boundaries require a class component; this is the one permitted
exception to the "functional components only" rule because React has no
function-component error boundary API.

ErrorBoundary API:
  <ErrorBoundary
    title="Something went wrong"
    message="This panel could not render."
    onReset={optionalCallback}
  >
    {children}
  </ErrorBoundary>

Behaviour:
  - Catch render-time errors in child components.
  - Render a compact fallback panel with:
      title
      message
      error message text
```

```
      "Try again" button if onReset is supplied
  - Log the error to console.error for developer visibility.
  - Do not swallow async errors from event handlers; those should be handled
locally.
  - Use tokens from src/ui/shared/tokens.js.

Wrap existing surfaces:
  - Model library/list area in App.jsx
  - ModelDetail content
  - Editors tab content
  - Execute panel content

Testing:
  - Unit/UI test: a child component that throws renders fallback text.
  - Test: onReset button invokes callback.
  - Test: normal child renders unchanged when no error is thrown.
```

Completion checklist:

- ~~`ErrorBoundary` exported from `shared/components.jsx`~~
- ~~Fallback UI uses tokens~~
- ~~Model library, ModelDetail, editors, and Execute wrapped~~
- ~~Reset callback supported~~
- ~~Error boundary tests pass~~

F5.2 — Model Export To JSON

Audit status: ✗ (models are Supabase-only; no file export)
Action: Add export button to existing ModelDetail
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.2 of Sprint 5.

Read these files before writing code:
  - src/ui/ModelDetail.jsx
  - src/ui/editors/index.jsx
  - src/engine/validation.js
  - docs/addition1_entity_model.md

Add a model JSON export action to the existing ModelDetail view.
Do not change the model schema.

Export behaviour:
  - Button label: "Export JSON"
  - Export the currently loaded model, including:
```

```
      name
      description if present
      model_json
      exportedAt ISO timestamp
      appVersion from package.json if practical, otherwise omit
  - Before export, run validateModel(model_json).
  - If validation has blocking errors, allow export only after showing a
warning:
      "This model has validation errors. Export anyway?"
  - Download a `.json` file named from the model name:
      des-studio-[model-name-slug].json
  - Use Blob + URL.createObjectURL.
  - Revoke the object URL after download.

UI rules:
  - Add the button near other ModelDetail actions.
  - Preserve existing Save/Back behaviour.
  - Do not add a new dependency.

Testing:
  - UI test: Export JSON button renders for a loaded model.
  - Unit/helper test: exported payload includes model_json and exportedAt.
  - Test: generated filename slug is stable.
  - Test: validation warning appears before exporting invalid model.
```

Completion checklist:

- ~~Export JSON button added to ModelDetail~~
- ~~Export payload includes model metadata and `model_json`~~
- ~~Filename generated from model name~~
- ~~Invalid model warning shown~~
- ~~Object URL revoked after download~~
- ~~Export tests pass~~

F5.3 — Model Import From JSON

Audit status: ✗ (no upload/import path)
Action: Add import affordance to model library
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.3 of Sprint 5.

Read these files before writing code:
  - src/App.jsx
  - src/db/models.js
```

```
  - src/engine/validation.js
  - docs/addition1_entity_model.md

Add model import from JSON to the existing model library.

Import behaviour:
  - Add "Import JSON" button near model library actions.
  - Use a hidden file input accepting .json and application/json.
  - Read the selected file with FileReader.
  - Accept both:
      exported payload from F5.2: { name, description, model_json, ... }
      raw model_json object
  - Validate the imported model_json with validateModel().
  - If blocking errors exist, show an error list and do not save.
  - If warnings only, show warning banner and allow import.
  - Imported model should be saved as a new private model owned by current
user.
  - Never preserve another user's user_id from imported JSON.
  - Never preserve is_public=true from imported JSON; default imported
models to private.
  - Suggested imported name:
      "[original name] (Imported)"
    with fallback "Imported model".

DB rules:
  - Use existing saveModel() wrapper or extend it.
  - saveModel() payload must include current user's user_id.
  - Do not call Supabase directly from App.jsx except through existing
wrappers.

Testing:
  - UI test: Import JSON button renders.
  - Test: valid exported payload calls saveModel with model_json and current
user_id.
  - Test: raw model_json import works.
  - Test: invalid JSON surfaces an error and does not call saveModel.
  - Test: imported payload does not preserve is_public=true or external
user_id.
```

Completion checklist:

- ~~Import JSON button added to model library~~
- ~~Exported payload and raw model_json supported~~
- ~~Imported model validates before save~~
- ~~Imported model is private and owned by current user~~
- ~~Invalid imports show errors and do not save~~
- ~~Import tests pass~~

F5.4 — Results Export

Audit status: ✗ (no results export from Execute panel)
Action: Add CSV/JSON export from Execute panel
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.4 of Sprint 5.

Read these files before writing code:
  - src/ui/execute/index.jsx
  - src/engine/statistics.js (from Sprint 4)
  - src/db/models.js

Add results export to the existing Execute panel.

Export formats:
  - JSON: complete latest results object, including experiment config,
    per-replication results, aggregate CI, seed, replications, warm-up, and
run duration.
  - CSV: flat table suitable for spreadsheets:
      replicationIndex, seed, served, reneged, avgWait, avgSvc, avgSojourn,
finalTime
    plus aggregate rows if available:
      metric, n, mean, lower95, upper95, halfWidth

UI behaviour:
  - Export buttons disabled until a run completes.
  - Labels:
      "Export Results JSON"
      "Export Results CSV"
  - File names:
      des-studio-results-[model-name-slug]-[timestamp].json
      des-studio-results-[model-name-slug]-[timestamp].csv
  - For single-run results, CSV should still contain one replication-style
row.
  - For cancelled/error runs, export only if partial results exist, and
filename should include "partial".

Testing:
  - Unit/helper test: JSON export payload contains experiment config and
results.
  - Unit/helper test: CSV contains expected headers.
  - Unit/helper test: multi-replication CSV includes one row per
replication.
  - UI test: export buttons disabled before run and enabled after run.
```

Completion checklist:

- [x] ~~JSON results export implemented~~
- [x] ~~CSV results export implemented~~
- [x] ~~Single-run and multi-run results supported~~
- [x] ~~Export buttons disabled before results exist~~
- [x] ~~Partial result export handled deliberately~~
- [x] ~~Results export tests pass~~

F5.5 — Fix Stats Panel Runs Count

Audit status: ~ (stats panel runs count always 0)
Action: Fix model/run stats derivation
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.5 of Sprint 5.

Read these files before writing code:
  - src/App.jsx
  - src/db/models.js
  - src/ui/execute/index.jsx

Find where the model library/card stats panel renders run count.
The audit notes runs count is always 0 because model.stats is not populated.

Fix behaviour:
  - Run count should reflect saved run history for that model.
  - If run history is already loaded in the client, derive the count from
it.
  - If DB wrapper support is needed, add a small function in
src/db/models.js,
    e.g. getRunStats(modelId, userId), but do not query Supabase directly
from UI.
  - Preserve user isolation:
      users see counts for their own model runs
      public model viewing must not leak other users' private run history
  - For forked public runs under ADR-002, counts belong to the fork owner.

Display:
  - Replace hardcoded/empty 0 with actual count.
  - Show a neutral placeholder such as "—" while loading if needed.
  - Do not block model list rendering if stats fail; show a non-fatal error
state.

Testing:
  - DB test: stats query filters by model_id and user/ownership as
appropriate.
```

```
  - UI test: model card renders non-zero run count when stats are provided.
  - UI test: loading/failed stats do not crash the model list.
```

Completion checklist:

- ~~Run count no longer always 0~~
- ~~Stats derived from run history or DB wrapper~~
- ~~User isolation preserved~~
- ~~Loading/failure states handled~~
- ~~DB/UI tests pass~~

F5.6 — Fix avg_service_time Column Mismatch (C9)

Audit status: ~ (`avg_service_time` column mismatch in DB wrapper/stats area)
Action: Fix `src/db/models.js` run/stat mapping
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.6 of Sprint 5.

Read these files before writing code:
  - src/db/models.js
  - src/ui/execute/index.jsx
  - any schema file present in the repo

Find the avg_service_time mismatch noted as C9.
Determine the actual column names expected by the schema and the actual
fields
returned by engine results.

Fix rules:
  - Do not rename database columns without an ADR and migration.
  - Prefer mapping engine summary.avgSvc to the existing DB column if
present.
  - If the DB column is absent or named differently, fix the wrapper
mapping,
    not the engine result shape.
  - Single-run and multi-replication batch results must both store service
time
    in results_json even if a scalar summary column is unavailable.
  - Keep backwards compatibility for existing run records.

Testing:
  - DB test: saveRun maps avgSvc/avg_service_time correctly based on current
schema wrapper.
  - DB test: missing avg service value does not throw.
```

```
  - UI test or helper test: run history displays avg service time from
either scalar column or results_json fallback.
```

Completion checklist:

- ~~Actual schema/wrapper mismatch identified~~
- ~~`saveRun()` maps avg service time correctly~~
- ~~Existing run records remain readable~~
- ~~Results_json includes avg service data~~
- ~~Tests pass~~

F5.7 — Keyboard Navigation + ARIA

Audit status: ✗ (keyboard/ARIA pass not yet done)
Action: Extend existing UI components
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.7 of Sprint 5.

Read these files before writing code:
  - src/App.jsx
  - src/ui/ModelDetail.jsx
  - src/ui/editors/index.jsx
  - src/ui/execute/index.jsx
  - src/ui/shared/components.jsx

Perform an accessibility pass focused on keyboard completion of a full
simulation workflow.

Required keyboard path:
  1. Navigate model library
  2. Open/create a model
  3. Move through editor tabs
  4. Edit form fields
  5. Save model
  6. Open Execute tab/panel
  7. Run simulation
  8. Read/export results

Implementation rules:
  - Add labels or aria-labels to icon-only buttons.
  - Ensure tab controls use button semantics and expose selected state.
  - Ensure destructive actions have accessible confirmation text.
  - Ensure form inputs have associated labels.
  - Ensure validation errors are programmatically associated where
```

```
practical.
  - Ensure focus is moved sensibly after modal/confirmation open and close.
  - Do not introduce a UI framework or accessibility dependency.
  - Preserve existing visual design.

Testing:
  - UI tests with @testing-library/user-event tab navigation for core flows.
  - Test: all primary buttons can be reached by keyboard.
  - Test: Execute Run button has accessible name and disabled state.
  - Test: validation error text is visible and discoverable.
  - Optional manual check: Lighthouse accessibility score >= 85.
```

Completion checklist:

- ~~Primary workflow keyboard reachable~~
- ~~Icon/destructive buttons have accessible names~~
- ~~Inputs have labels~~
- ~~Tabs expose selected state~~
- ~~Error text visible and associated where practical~~
- ~~Accessibility UI tests pass~~

F5.8 — First-Run Onboarding

Audit status: ✗ (no onboarding/sample model)
Action: Add welcome panel + sample M/M/1 model
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.8 of Sprint 5.

Read these files before writing code:
  - src/App.jsx
  - src/db/models.js
  - docs/addition1_entity_model.md
  - tests/engine/mm1_benchmark.js

Add first-run onboarding for users with no models.

Onboarding behaviour:
  - If the authenticated user's model list is empty, show a compact welcome
panel.
  - The first screen should still be the usable app/model library, not a
marketing landing page.
  - Offer actions:
      "Create blank model"
      "Create sample M/M/1 model"
```

```
      "Import JSON"
  - The sample model should be a valid M/M/1 queueing model compatible with
current schema.
  - Sample model should pass validateModel().
  - Saving the sample model creates a private model owned by the current
user.
  - Do not show onboarding once the user has models.

Content rules:
  - Keep copy brief and practical.
  - Do not include tutorial paragraphs or keyboard shortcut explanations in-
app.
  - Use existing tokens and model card/list layout.

Testing:
  - UI test: empty model list shows onboarding actions.
  - UI test: non-empty model list does not show onboarding.
  - Test: Create sample M/M/1 calls saveModel with current user_id.
  - Engine/validation test: sample model validates and can run.
```

Completion checklist:

- ✅ ~~Empty model list shows onboarding panel~~
- ✅ ~~Blank model, sample model, and import actions available~~
- ✅ ~~Sample M/M/1 model validates~~
- ✅ ~~Sample model saves as private and owned by current user~~
- ✅ ~~Onboarding hidden for users with models~~
- ✅ ~~Onboarding tests pass~~

F5.9 — Model Delete (Owner Only)

Audit status: ✗ (deleteModel() exists in db/models.js — no UI surface anywhere)
Existing code: `App.jsx` (ModelCard), `src/db/models.js`
Action: New UI affordance + ownership guard
Status: ✅ | **Completed:** 2026-05-04

```
Task F5.9 of Sprint 5.

Read these files before writing code:
  - src/App.jsx
  - src/db/models.js
  - schema file if present
  - docs/decisions/ADR-001-auth-model.md
  - docs/decisions/ADR-002-public-model-runs.md
```

```
Add a Delete button to ModelCard:
  - Visible only when model.user_id === currentUser.id.
  - Not rendered on public models owned by other users.
  - On click, show confirmation dialog:
      "Delete '[model name]'? This cannot be undone."
  - On confirm, call deleteModel(model.id, userId).
  - On success, remove model from local state without page reload.
  - On error, display error banner.

Extend deleteModel() in db/models.js:
  - Require both model id and userId.
  - If either is missing, return structured error and do not execute
Supabase query.
  - Add .eq('user_id', userId) to delete query.
  - This enforces ownership at the application layer in addition to Supabase
RLS.
  - If no row deleted or Supabase reports an ownership/id mismatch, return
structured error.

Document cascade behaviour:
  - Read schema file if present and confirm whether run records have ON
DELETE CASCADE from models.
  - If yes, note this in a concise code comment in deleteModel().
  - If no, add a finding in the Sprint 5 notes: orphaned run records are a
data integrity risk.

Testing:
  - UI test: owner sees Delete button on their own model.
  - UI test: non-owner does not see Delete button on a public model.
  - UI test: confirmation dialog appears before delete executes.
  - UI test: deleteModel is not called if user cancels.
  - DB test: deleteModel query includes .eq('user_id', userId).
  - DB test: deleteModel called without userId does not execute.
```

Completion checklist:

- [x] ~~Delete button visible to owner only~~
- [x] ~~Confirmation dialog shown before delete~~
- [x] ~~`deleteModel()` includes `owner_id` ownership guard (current schema field; prompt's `user_id` reference is stale)~~
- [x] ~~Missing userId/id blocks delete query~~
- [x] ~~Cascade behaviour checked: no committed schema file confirms `simulation_runs` cascade; orphaned run records remain a data integrity risk to resolve with the next schema migration~~
- [x] ~~UI test passes~~
- [x] ~~DB layer test passes~~

Sprint 6 — LLM Integration & Results Analysis

Goal: Embed an AI assistant panel into the Execute view that interprets simulation results in natural language, compares scenarios, and provides sensitivity commentary. All LLM calls route through a Supabase Edge Function proxy backed by the Anthropic API; no API keys are exposed in the browser bundle.

Status: ✅ Complete | **Started:** 2026-05-04 | **Completed:** 2026-05-04
Prerequisite: Sprint 5 exit gate passed. Results export (F5.4) must produce structured JSON that can be passed to the LLM prompt.

Feature	Audit Status	Action
F6.1 — AI assistant panel	✗	New: collapsible right-hand panel in Execute view
F6.2 — KPI narrative generation	✗	New: prompt builder + `llm-proxy` API client
F6.3 — Scenario comparison (A vs B)	✗	New: multi-run selector + comparative prompt
F6.4 — Sensitivity commentary	✗	New: replication variance summary fed to LLM
F6.5 — Streaming response display	✗	New: streamed token display in assistant panel

Design Principles for Sprint 6

The LLM integration must never modify the model or engine. It is read-only and advisory. The browser builds prompts and handles responses, but every provider call is routed through the Supabase Edge Function `llm-proxy`. The proxy holds `ANTHROPIC_API_KEY` server-side, pins the allowed model, and streams responses back to the client. The results JSON passed to the LLM is the same structured object produced by `buildEngine()` — no intermediate transformation layer is required.

Context injected into every prompt:

- Model name and description
- Experiment configuration (warm-up, run duration, replications, seed)
- KPI summary: mean wait times per queue, resource utilisation, throughput, renege rates
- Confidence intervals (if N > 1 replications)
- (For comparison prompts) side-by-side KPIs for two named runs

Sprint 6 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 6 of DES Studio.

Sprint 6 adds an LLM-powered AI assistant panel to the Execute view.
The assistant interprets simulation results, compares scenarios, and
provides sensitivity commentary using the Supabase `llm-proxy` Edge
Function backed by the Anthropic API (claude-sonnet).

Completed in Sprints 1–5: [paste]

Features:
  F6.1 — Collapsible AI assistant panel (right of Execute view)
  F6.2 — KPI narrative: LLM reads results JSON, returns plain-English
interpretation
  F6.3 — Scenario comparison: user selects two runs; LLM produces A-vs-B
narrative
  F6.4 — Sensitivity commentary: replication CI data fed to LLM for variance
analysis
  F6.5 — Streaming response display using Anthropic streaming API

CRITICAL:
  - LLM never modifies model or engine state — read-only, advisory
  - API key must NOT be embedded in client code — use Supabase Edge Function
as proxy
  - Prompt templates must be versioned in src/llm/prompts.js (new file)
  - All LLM calls are cancellable — user can interrupt streaming response
  - If Anthropic API is unavailable, panel shows graceful fallback (no
crash)

Definition of done:
  - User runs a model, opens AI panel, clicks "Explain results" — receives
narrative
  - User selects two saved runs, clicks "Compare" — receives A-vs-B analysis
  - Streaming tokens appear progressively (not batch-loaded at end)
  - npm test -- --run passes (new unit tests for prompt builder)
  - npm run build succeeds
```

F6.1 — AI Assistant Panel

Audit status: ✗ (not implemented)
Action: New collapsible panel — extends `execute/index.jsx`
Status: ✅ | **Completed:** 2026-05-04

```
Task F6.1 of Sprint 6.

Read src/ui/execute/index.jsx — understand the current panel layout.
```

```
Add a collapsible AI assistant panel to the right side of the Execute view:
  - Toggle button: "AI Insights" (shows/hides panel)
  - Panel width: 320px, full height of Execute view
  - Initial state: collapsed (does not affect existing layout)
  - When expanded: existing Execute content shrinks to fill remaining width
  - Panel header: "AI Assistant" with a close (×) button
  - Panel body: three sections
      1. "Explain results" button (active only after a run completes)
      2. "Compare runs" – dropdown to select a second run from run history
      3. Response area: scrollable text with streaming token display

No LLM calls in this task. This task builds the UI shell only.
The response area shows a placeholder: "Run the model to generate insights."

Write a UI test: panel toggles correctly; buttons disabled before run.
```

Completion checklist:

- ~~AI panel renders and toggles without affecting existing Execute layout~~
- ~~Buttons disabled before first run completes~~
- ~~Placeholder text shown in response area~~
- ~~UI test passes~~

F6.2 — KPI Narrative Generation

Audit status: ✗ (not implemented)
Action: New `src/llm/` directory — `prompts.js` + `apiClient.js`
Status: ✅ | **Completed:** 2026-05-04

```
Task F6.2 of Sprint 6.

Create src/llm/prompts.js (new file):
  Export buildNarrativePrompt(model, experimentConfig, results):
    - System prompt: "You are an expert simulation analyst. Interpret
      the following discrete-event simulation results for a non-specialist
      audience. Be concise – 150–200 words. Use plain English."
    - User message: structured JSON block containing:
        model.name, model.description
        experiment: { warmup, runDuration, replications, seed }
        kpis: { queues: [{name, meanWait, maxWait, renegeRate}],
                resources: [{name, utilisation, busyCount}],
                throughput, totalEntities }
    - Instruction: "Highlight the most significant findings. Flag any
      queues where mean wait exceeds 2 × service time (possible overload)."

Create src/llm/apiClient.js (new file):
```

```
  Export streamNarrative(prompt, onToken, onComplete, onError):
    - POST to /functions/v1/llm-proxy (Supabase Edge Function — see below)
    - Handle SSE streaming: call onToken(text) for each chunk
    - Call onComplete() when stream ends
    - Call onError(err) on failure

Create supabase/functions/llm-proxy/index.ts (new Supabase Edge Function):
  - Receives { messages, model, max_tokens } from client
  - Forwards to Anthropic API using ANTHROPIC_API_KEY secret (server-side)
  - Streams response back to client
  - Validates that model is claude-sonnet-4-20250514 only (no other models)
  - Rate-limits: max 10 requests per user per hour (uses Supabase user JWT)

In the AI panel (F6.1): wire "Explain results" button to streamNarrative().
Display streaming tokens in the response area as they arrive.

Unit tests (tests/llm/prompts.test.js):
  - buildNarrativePrompt returns valid JSON in user message
  - Overloaded queue (meanWait > 2× serviceTime) is flagged in prompt
  - Prompt does not exceed 2000 tokens (count words × 1.3 heuristic)
```

Completion checklist:

- ~~`src/llm/prompts.js` with `buildNarrativePrompt()` exported~~
- ~~`src/llm/apiClient.js` with `streamNarrative()` exported~~
- ~~Supabase Edge Function `llm-proxy` source added locally~~
- ~~Supabase Edge Function `llm-proxy` deployed~~
- ~~API key never appears in client-side code~~
- ~~Streaming tokens display progressively in panel~~
- ~~Unit tests pass~~

F6.3 — Scenario Comparison (A vs B)

Audit status: ✗ (not implemented)
Action: Extend `src/llm/prompts.js`; extend AI panel
Status: ✅ | **Completed:** 2026-05-04

```
Task F6.3 of Sprint 6.

Read src/llm/prompts.js (from F6.2).

Add buildComparisonPrompt(modelName, runA, runB) to prompts.js:
  runA and runB are results objects from two different simulation runs.
  Each has: experimentConfig, kpis, run label (user-assigned or timestamp).
```

```
  System prompt: "You are an expert simulation analyst. Compare the two
  simulation runs below and explain the key differences to a non-specialist.
  Be concise — 200–250 words. Use a Before/After or Option A/Option B
frame."

  User message: structured JSON with runA and runB side by side.

In the AI panel:
  - "Compare runs" section: primary run is the most recent completed run
  - Dropdown: select a second run from run history (last 20 runs)
  - "Compare" button: builds comparison prompt and calls streamNarrative()
  - Response replaces the narrative area content

Unit tests (add to tests/llm/prompts.test.js):
  - buildComparisonPrompt includes both run labels in output
  - Produces valid JSON structure
```

Completion checklist:

- ~~`buildComparisonPrompt()` exported from `prompts.js`~~
- ~~Comparison prompt includes both run KPIs side by side~~
- ~~AI panel dropdown populated from completed replication rows in the current Execute session~~
- ~~Streaming narrative displayed for comparison~~
- ~~Unit tests pass~~

F6.4 — Sensitivity Commentary

Audit status: ✗ (not implemented)
Action: Extend `src/llm/prompts.js`; extend AI panel
Status: ✅ | **Completed:** 2026-05-04

```
Task F6.4 of Sprint 6.

This feature is only available when N >= 5 replications have been run
(confidence intervals are meaningful only with sufficient sample size).

Add buildSensitivityPrompt(modelName, experimentConfig, ciResults) to
prompts.js:
  ciResults: array of KPIs with { name, mean, ci95Lower, ci95Upper, stdDev }

  System prompt: "You are an expert simulation analyst. Explain the
  statistical uncertainty in the following simulation results. Identify
  which KPIs have wide confidence intervals and what this implies for
  decision-making. Be concise — 150–200 words."
```

```
In the AI panel:
  - "Sensitivity" button: shown only when N >= 5 replications exist
  - Calls buildSensitivityPrompt and streamNarrative()

Unit test: sensitivity button disabled when replications < 5.
```

Completion checklist:

- ~~`buildSensitivityPrompt()` exported from `prompts.js`~~
- ~~Sensitivity button disabled when N < 5~~
- ~~CI data correctly structured in prompt~~
- ~~Unit test passes~~

F6.5 — Streaming Response Display

Audit status: ✗ (not implemented)
Action: Extend AI panel component; extend `apiClient.js`
Status: ✅ | **Completed:** 2026-05-04

```
Task F6.5 of Sprint 6.

Ensure the streaming response in the AI panel meets production quality:
  - Tokens appear character-by-character (or chunk-by-chunk per SSE event)
  - A "Stop" button cancels the in-flight request using AbortController
  - A loading spinner appears while waiting for first token
  - If the stream errors: display "Analysis unavailable — try again" (amber
banner)
  - Copy-to-clipboard button appears after stream completes
  - Response is formatted safely: render plain text or a minimal React-
rendered
    markdown subset (paragraphs, lists, bold/italic). Do not use
    dangerouslySetInnerHTML. Do not add a markdown dependency unless
approved
    by an ADR or explicit implementation decision.

Unit test: onError callback triggered when fetch rejects; panel shows error
banner.
```

Completion checklist:

- ~~Streaming tokens display progressively~~
- ~~Stop button cancels stream via `AbortController`~~
- ~~Loading spinner shown before first token~~

- [x] ~~Error banner shown on failure (no crash)~~
- [x] ~~Copy button present after completion~~
- [x] ~~Response rendered safely (no `dangerouslySetInnerHTML`; no new markdown dependency without approval)~~
- [x] ~~Unit test passes~~

Sprint 6 Completion Gate

```
npm test -- --run                          # Zero failures
npm test -- llm                            # LLM prompt builder tests pass
npm test -- ui                             # AI panel toggle test passes
npm run build                              # Succeeds
# Manual: open Execute panel, run model, click "AI Insights", verify
narrative appears
# Manual: select two runs, click Compare, verify comparison narrative
appears
```

Local gate result (2026-05-04):

- `npm test` — 32 files, 343 tests passed
- `npm test -- llm execute-panel` — 2 files, 14 tests passed
- `npm run build` — succeeds
- Hosted Supabase `llm-proxy` deployed to project `znkknldzdfajcrpabtmg`

Sprint 6 Refinements Identified

These refinements came from local UI review after Sprint 6 was implemented.

Refinement	Status	Notes
Compare against saved run history, not only current-session replication rows	✅ Implemented 2026-05-04	AI comparison dropdown now loads saved runs for the current model from `simulation_runs`, with current-session replications retained as fallback options.
Add user-facing run labels	✅ Implemented 2026-05-05	Execute now captures an optional run label, persists it in `results_json`, and uses it in the AI comparison dropdown and History table.
Export options from History page	✅ Implemented 2026-05-05	History now offers JSON and CSV exports for normalized saved runs, including user-facing run labels.
Fix low-contrast termination mode labels	✅ Implemented	Execute termination radio labels now use the shared text color and mono font

Refinement	Status	Notes
	2026-05-05	for readability.
Improve AI panel layout at narrow widths	🔄 Partial	Toolbar clipping fixed in commit `667b46a`; panel could still become a drawer or stack below results on small screens.
Decide whether AI analyses should be persisted	🟪 Future	Current responses are session-only. Persisting analysis notes would need a product decision and schema/API work.
Improve prompt grounding with richer per-queue/per-resource KPIs	🟪 Future	Current prompts use the available summary/run data. Better queue/resource breakdowns should come from richer result payloads.
Show more specific proxy/configuration errors	🟪 Future	Current UI shows a graceful generic unavailable message. Deployment diagnostics could distinguish missing proxy, missing secret, and provider failure.
Harden production rate limiting	🟪 Future	`llm-proxy` currently uses simple in-memory request counting. Persistent user-scoped rate limiting would be stronger.
Add a manual LLM deployment checklist	🟪 Future	Checklist should cover CLI login, secret setting, function deploy, dashboard verification, and one real AI request.

Sprint 7A — Platform Foundation: Roles, Settings & TypeScript

Goal: Establish the first platform foundations before further feature expansion: user/admin roles, durable user settings, and a decision on whether to adopt TypeScript incrementally.

Status: ✅ Complete | **Started:** 2026-05-05 | **Completed:** 2026-05-05
Prerequisite: Sprint 6 exit gate passed. This sprint should be completed before Sprint 8 AI authoring and preferably before Sprint 7 schema-heavy time-varying work.

Feature	Audit Status	Action
F7A.1 — Authorization model v2	✅	Accepted: `profiles.role` stores platform role (`user`, `admin`); model owner/editor/viewer access remains separate

Feature	Audit Status	Action
F7A.2 — User settings strategy	✅	Accepted: durable preferences belong in a dedicated `user_settings` table with owner-only access by default
F7A.3 — TypeScript reassessment	✅	Accepted: retire JavaScript-only rule and adopt TypeScript incrementally at schema/domain boundaries
F7A.4 — Documentation and planning updates	✅	ADR-008, ADR-009, CLAUDE.md, and roadmap updated; follow-on decisions deferred explicitly

Design Principles for Sprint 7A

- Keep this sprint deliberately narrow: roles/admin, user settings, and TypeScript decision only.
- Do not implement a partial SaaS tenancy model by accident. Tenancy/workspaces remain a follow-on architecture decision.
- Do not implement LLM provider switching in this sprint. Provider-neutral LLM architecture remains a follow-on architecture decision.
- Do not begin a TypeScript rewrite. ADR-009 is about incremental adoption and boundary-first migration.
- Preserve the build-on rule. Architecture changes should reduce risk without rewriting working features.

Sprint 7A Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 7A of DES Studio.

Sprint 7A is a narrow architecture-readiness sprint before further feature
expansion.
It addresses:
  1. user/admin roles
  2. durable user settings
  3. TypeScript reassessment

Read:
  - CLAUDE.md
  - docs/DES_Studio_Build_Plan.md
  - docs/decisions/ADR-001-auth-model.md
  - docs/decisions/ADR-002-public-model-runs.md
  - docs/decisions/ADR-008-platform-roles-saas-llm-configuration.md
  - docs/decisions/ADR-009-typescript-reassessment.md
  - src/App.jsx
  - src/db/models.js
```

```
Tasks:
  F7A.1 — Propose authorization model v2 with platform admin role and model-
level access implications
  F7A.2 — Propose durable user settings strategy and first persistence shape
  F7A.3 — Reassess JavaScript-only rule and recommend accept/reject for
incremental TypeScript adoption
  F7A.4 — Update ADRs, CLAUDE.md, and this build plan with accepted
decisions and deferred questions

Definition of done:
  - ADR-008 is revised to cover roles/admin and user settings as the
accepted near-term scope
  - ADR-009 is accepted or rejected with rationale
  - SaaS tenancy, LLM provider switching, architecture health review, and
Execute UX are explicitly deferred
  - CLAUDE.md Current Sprint and Future-Claude notes reflect the decisions
  - No product feature implementation is required unless a small
documentation helper is needed
```

Sprint 7A Completion Gate

```
# Documentation/architecture sprint: no code build required unless code is
changed.
git diff --check                        # Succeeds
# Manual: confirm ADR register and Open Architectural Decisions agree
# Manual: confirm Sprint 7, Sprint 8, and Sprint 9 prerequisites reflect
accepted decisions
```

Sprint 7A Completion Notes

- ADR-008 accepted. Platform roles use `profiles.role`; initial values are `user` and `admin`.
- Model-level ownership/access remains unchanged: `owner_id`, `access`, and `visibility` continue to govern model permissions.
- User settings should use a dedicated `user_settings` table, not local storage or display-profile fields.
- ADR-009 accepted. TypeScript is allowed incrementally, but no broad conversion should occur without a migration task.
- SaaS tenancy/workspaces, LLM provider switching, architecture health review, and Execute UX redesign remain deferred follow-on work.

Post-7A Sequencing Decision

Decision: implement a small platform foundation sprint now, before Sprint 7, but defer broader platform/SaaS work until after the modelling roadmap checkpoints.

Rationale:

- Sprint 7 changes model schema and distribution/runtime structures. Adding TypeScript tooling and first domain contracts before that work reduces schema drift risk.
- Roles and user settings are low-scope persistence foundations that future admin/settings surfaces can build on without interrupting Sprint 7.
- Full SaaS tenancy, LLM provider switching, and Execute UX redesign are larger product/architecture topics. They should not block time-varying modelling features unless their absence creates a direct implementation risk.

Sequence:

Order	Sprint	Implement now?	Reason
1	Sprint 7B — Platform Foundation Implementation	Yes, before Sprint 7	Small, low-risk persistence/type foundation from accepted ADRs
2	Sprint 7 — Dynamic Distributions & Time-Varying Resources	Yes, after 7B	Schema-heavy modelling work benefits from typed contracts
3	Sprint 8A — LLM Provider Architecture Preflight	Yes, before Sprint 8	AI model authoring should not deepen Anthropic-only coupling
4	Sprint 8 — AI Generated Model Authoring	Yes, after 8A	Builds second authoring mode over canonical model JSON
5	Sprint 9A — Visual Designer Architecture Preflight	Optional before Sprint 9	Decide canvas dependency and graph metadata if still open
6	Sprint 9 — Visual Designer Authoring	Yes, after preflight	Builds third authoring mode
Later	SaaS tenancy/workspaces	No, defer	Needs separate schema/RLS/billing-oriented architecture pass
Later	Execute running-model UX redesign	No, defer	Important, but deserves a dedicated design/refinement sprint

---

Sprint 7B — Platform Foundation Implementation

Goal: Implement the minimal platform foundation accepted in Sprint 7A without building full admin UI, SaaS tenancy, or broad TypeScript migration.

Status: ✅ Complete | **Started:** 2026-05-05 | **Completed:** 2026-05-05
Prerequisite: Sprint 7A exit gate passed. ADR-008 and ADR-009 accepted.

Feature	Audit Status	Action
F7B.1 — Profiles role migration	✅	Added Supabase migration for `profiles.role` default/check constraint and admin helper
F7B.2 — User settings table	✅	Added `user_settings` migration with owner-only RLS and updated timestamp trigger
F7B.3 — Settings DB wrappers	✅	Added `fetchUserSettings()` and `saveUserSettings()` through DB layer
F7B.4 — App profile role normalization	✅	Normalized platform roles and exposed `isAdmin` without changing model permissions
F7B.5 — TypeScript tooling baseline	✅	Added TypeScript dependency, `tsconfig.json`, and `npm run typecheck`
F7B.6 — First typed contracts	✅	Added typed contracts for canonical model JSON, run/result payloads, and user settings
F7B.7 — Tests	✅	Added DB wrapper/role/settings tests and contract tests

Design Principles for Sprint 7B

- Do not build a full admin UI.
- Do not implement SaaS tenants/workspaces.
- Do not convert large UI files to TypeScript.
- Do not give admins implicit edit rights over private user models.
- Keep TypeScript adoption boundary-first: contracts and tooling only.

Sprint 7B Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 7B of DES Studio.

Sprint 7B implements the small platform foundation from Sprint 7A.

Read:
  - CLAUDE.md
  - docs/DES_Studio_Build_Plan.md
  - docs/decisions/ADR-008-platform-roles-saas-llm-configuration.md
  - docs/decisions/ADR-009-typescript-reassessment.md
  - src/App.jsx
```

```
  - src/db/models.js
  - tests/db/models.test.js
  - tests/setup.js

Tasks:
  F7B.1 — Add schema/migration notes for profiles.role and user_settings
  F7B.2 — Add DB wrappers for fetchUserSettings() and saveUserSettings()
  F7B.3 — Normalize profile role and expose isAdmin in App state/overrides
without changing model permissions
  F7B.4 — Add TypeScript tooling baseline with allowJs and a type-check
command
  F7B.5 — Add first typed contract files for model JSON and run/result
payloads
  F7B.6 — Add focused tests for settings wrappers, role normalization, and
contract exports

Definition of done:
  - No direct Supabase settings calls from UI components
  - Admin role is detected but does not bypass model owner/editor access
  - User settings persistence is represented in DB wrappers and tests
  - TypeScript tooling is present but no broad conversion has occurred
  - npm test passes for focused DB/UI tests
  - npm run build succeeds
```

Sprint 7B Completion Gate

```
npm test -- db onboarding model-export contracts  # 4 files, 33 tests passed
npm run build                                     # Succeeds
npm run typecheck                                 # Succeeds
# Manual: verify regular user profile still loads and model permissions are
unchanged
```

Sprint 7B Completion Notes

- No full admin UI was added.
- No SaaS tenancy/workspace model was added.
- No large UI file was converted to TypeScript.
- Admin status is exposed as `profile.isAdmin` / `isAdmin` only; model permissions still derive from owner/editor access.
- `npm install --save-dev typescript` reported four moderate audit findings in existing dependency tree context; no audit fix was applied because that can introduce unrelated breaking changes.
- `npm audit --audit-level=moderate` confirms the four findings are via `esbuild <=0.24.2`, `vite <=6.4.1`, `vite-node <=2.2.0-beta.2`, and `vitest` dependency chains. The suggested fix requires `npm audit fix --force` and a breaking Vite 8 upgrade, so it is deferred to dependency maintenance.

- The Supabase migration `supabase/migrations/20260505073000_platform_roles_user_settings.sql` is committed locally and pushed, but must still be applied to the linked Supabase project before any UI depends on `user_settings`.

Sprint 7 — Dynamic Distributions & Time-Varying Resources

Goal: Enable modellers to represent systems where arrival rates and resource capacity change over simulated time. This is essential for realistic models of emergency departments, call centres, and any system with diurnal or shift-based patterns.

Status: ✅ Complete | **Started:** 2026-05-05 | **Completed:** 2026-05-05
Prerequisite: Sprint 7B exit gate passed. Sprint 7 may proceed with ADR-009 in force: schema-heavy work should use or extend typed model contracts where practical, but broad TypeScript conversion is still out of scope.

Architectural constraint: All time-varying logic is scheduled as B-Events. No new Phase types are introduced. The Three-Phase engine (`engine/index.js`) is not restructured — it is extended via the existing B-Event mechanism.

Feature	Audit Status	Action
F7.1 — Piecewise inter-arrival distribution schema	✅	Added `Piecewise` registry entry, aliases, active-period sampler, and tests
F7.2 — NHPP arrival B-Event scheduler	✅	RATE_CHANGE events are scheduled at init and piecewise sampling is clock-aware
F7.3 — Server shift schedule schema	✅	Added `shiftSchedule[]` to typed contracts and schema docs
F7.4 — Shift change B-Event handler	✅	SHIFT_CHANGE events scale server instances and warn when busy capacity exceeds target
F7.5 — Time-varying distribution picker UI	✅	Extended shared and editor DistPicker controls with non-recursive piecewise period editing
F7.6 — Shift schedule editor UI	✅	Added server shift schedule section in the entity editor
F7.7 — Engine validation for time-varying params	✅	Added V12–V15 validation and focused tests

Design Principles for Sprint 7

Piecewise inter-arrival rates. The modeller defines an ordered list of `{ startTime, distribution }` pairs. At each transition time, the engine schedules a `RATE_CHANGE` B-Event as an explicit transition marker in the FEL. Sampling is clock-aware: whenever a schedule delay is sampled, the `Piecewise` sampler chooses the period active at the current simulation clock and delegates to that period's distribution. This mechanism requires no changes to Phase B or Phase C — only the existing B-Event scheduling path and sampler context.

```
PIECEWISE DISTRIBUTION SCHEMA
{ "type": "piecewise",
  "periods": [
    { "startTime": 0,   "distribution": { "type": "exponential", "rate": 0.5
} },
    { "startTime": 480, "distribution": { "type": "exponential", "rate": 1.5
} },
    { "startTime": 960, "distribution": { "type": "exponential", "rate": 0.8
} }
  ]
}
```

Resource shift schedules. In the current model, resources are represented by server entity types (`role: "server"`), not a separate Resource node table. The modeller defines capacity steps on a server entity type: `{ time, capacity }` pairs. Each step is pre-scheduled as a `SHIFT_CHANGE` B-Event at model initialisation. Capacity maps onto server entity instances: increases create idle server instances; decreases retire idle excess only; busy excess servers complete naturally and add a warning to run results.

```
SHIFT SCHEDULE SCHEMA (on server entity type)
{ "shiftSchedule": [
    { "time": 0,    "capacity": 3 },
    { "time": 480,  "capacity": 6 },
    { "time": 960,  "capacity": 2 }
  ]
}
```

Sprint 7 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 7 of DES Studio.

Sprint 7 adds time-varying arrival rates and resource shift schedules.
Both features are implemented as pre-scheduled B-Events — no new Phase
types or engine restructuring.

Completed in Sprints 1–5: [paste]

Features:
```

```
  F7.1 — Piecewise distribution type: new registry entry in distributions.js
  F7.2 — NHPP arrival scheduler: RATE_CHANGE B-Events scheduled at model
init
  F7.3 — Shift schedule schema: new shiftSchedule[] property on server
entity types
  F7.4 — Shift change B-Event handler: applies server capacity at scheduled
time
  F7.5 — Time-varying distribution picker UI: extends existing DistPicker
  F7.6 — Shift schedule editor: extends existing server entity type
configuration panel
  F7.7 — Validation: piecewise periods must be time-ordered; shift times
must be
          within run duration; capacity values must be positive integers

CRITICAL:
  - The Three-Phase engine loop is NOT restructured
  - RATE_CHANGE and SHIFT_CHANGE B-Events are scheduled at buildEngine()
init
  - Server capacity changes take effect on the next Phase C scan (correct
behaviour)
  - Seeded RNG must be used for all sampling in piecewise distributions
  - A piecewise distribution with a single period is equivalent to a static
distribution
    (no special-casing needed)
  - Warm-up reset does NOT reset piecewise period or shift schedule position
    (the schedule is time-indexed, not statistics-indexed)

Definition of done:
  - M/M/1 with time-varying arrival passes: high-rate period produces longer
queues
  - Shift schedule test: capacity drops from 3 to 1 at T=500, queue grows
correctly
  - All existing engine tests still pass
  - npm test -- --run passes
```

F7.1 — Piecewise Distribution Schema

Audit status: ✗ (distributions.js supports 7 types — piecewise is absent)
Action: Extend `distributions.js` registry — no other engine files change
Status: ⬜ | **Completed:** —

```
Task F7.1 of Sprint 7.

Read src/engine/distributions.js — show me the current DISTRIBUTIONS
registry
object and the full list of currently registered types.
```

```
Add a new Piecewise distribution entry (or `piecewise` if the registry has
already been normalized by implementation time). Preserve the existing
distribution naming convention in `DISTRIBUTIONS`:
  Schema: { "type": "piecewise", "periods": [{ "startTime": number,
"distribution": Distribution }] }

  Validation function (runs at validateModel() time):
    - periods must be a non-empty array
    - periods must be sorted ascending by startTime
    - periods[0].startTime must be 0 (simulation must have a rate from T=0)
    - each period.distribution must be a valid registered distribution
    - periods must not overlap (guaranteed by sort + uniqueness check on
startTime)

  Sample function (accepts T_now from engine context):
    - Identify the active period: largest startTime <= T_now
    - Delegate to the active period's distribution sampler
    - The 'piecewise' sampler does NOT itself schedule RATE_CHANGE events —
      that is the NHPP scheduler's responsibility (F7.2)

Add to tests/engine/distributions.test.js:
  - Piecewise with two periods returns correct distribution for each time
range
  - Piecewise with period[0].startTime != 0 fails validation
  - Piecewise with unsorted periods fails validation
  - Sample at T_now in period 1 uses period 1 distribution
```

Completion checklist:

- ~~`piecewise` registered in `distributions.js`~~
- ~~Validation: startTime=0, sorted, non-overlapping~~
- ~~Sampler delegates to active period by T_now~~
- ~~All new distribution tests pass~~
- ~~Existing distribution tests unchanged~~

F7.2 — NHPP Arrival B-Event Scheduler

Audit status: ✗ (ARRIVE always uses a static distribution)
Action: Extend B-Event scheduling in `phases.js`, ARRIVE macro path in `macros.js`, and `engine/index.js` init
Status: ⬜ | **Completed:** —

```
Task F7.2 of Sprint 7.
```

```
Read src/engine/phases.js — show me how B-Events are scheduled.
Read src/engine/macros.js — show me the ARRIVE macro in full.
Read src/engine/index.js — show me the engine initialisation block.

Arrival B-Events currently use static schedule distributions and ARRIVE
macro
effects. For piecewise distributions, engine initialization must
additionally
schedule RATE_CHANGE B-Events at period transition times.

Extend engine initialisation in buildEngine():
  For each arrival B-Event or schedule row with a piecewise inter-arrival
distribution:
    For each period transition (periods[i].startTime where i > 0):
      Schedule a RATE_CHANGE B-Event at startTime
      Payload: { eventId or scheduleRowId, newDistribution:
periods[i].distribution }

Add RATE_CHANGE B-Event handler to phases.js:
  When fired: update the active distribution reference for that arrival
schedule
  No other state changes — the next ARRIVE B-Event will use the new
distribution

Write a unit test:
  - Model: single source, piecewise dist with period at T=0 (rate=0.5) and
T=100 (rate=2.0)
  - Run to T=200
  - Assert: mean inter-arrival in [0,100) is close to 1/0.5 = 2.0
  - Assert: mean inter-arrival in [100,200) is close to 1/2.0 = 0.5
  - Tolerance: 10% (stochastic — use large N)
```

Completion checklist:

- [x] ~~RATE_CHANGE B-Events scheduled at `buildEngine()` init for piecewise sources~~
- [x] ~~RATE_CHANGE handler records the transition marker; sampling remains clock-aware~~
- [x] ~~Schedule sampling uses the currently-active distribution at time of sampling~~
- [x] ~~Unit test demonstrates clock-based period selection~~
- [x] ~~All existing engine tests pass~~

F7.3 — Server Shift Schedule Schema

Audit status: ✗ (server entity types have no shiftSchedule property)
Action: Extend entity model schema; extend `docs/addition1_entity_model.md`
Status: ⬜ | **Completed:** —

```
Task F7.3 of Sprint 7.

Read docs/addition1_entity_model.md Section 3.1 (Resource State Variables)
and the current entity type schema. Confirm whether the current schema uses
server entity types (`role: "server"`) rather than separate Resource nodes.

Add shiftSchedule as an optional property on server entity types:
  Schema: { "shiftSchedule": [{ "time": number, "capacity": number }] }
  Default: absent (no shift schedule = static capacity throughout run)

Update addition1_entity_model.md Section 3.1 to document shiftSchedule.

Update the model JSON schema (if a formal schema file exists — check docs/).

Validation rules to add to validateModel() (F7.7):
  V12 — If shiftSchedule is present:
    - periods must be sorted ascending by time
    - periods[0].time must be 0
    - all capacity values must be positive integers
    - all times must be within run duration (if run duration is set)
    - the first period sets the initial server capacity (overrides the
static
      count/capacity field if shiftSchedule[0] is present)

No engine or UI changes in this task — schema and validation spec only.
```

Completion checklist:

- ~~`shiftSchedule` schema documented for server entity types in `addition1_entity_model.md`~~
- ~~Validation rules V12–V15 added to `validateModel()` spec in `addition1_entity_model.md`~~
- ~~Model JSON contract updated in `src/contracts/model.ts`~~

F7.4 — Shift Change B-Event Handler

Audit status: ✗ (no SHIFT_CHANGE B-Event exists)
Action: Extend `phases.js` and `engine/index.js` init
Status: ⬜ | **Completed:** —

```
Task F7.4 of Sprint 7.

Read src/engine/phases.js and src/engine/index.js.

ARCHITECTURAL DECISION REQUIRED before coding:
```

```
  The current engine pre-creates server entities from entityTypes[].count.
  Confirm whether shift capacity should be implemented by:
    A. Creating/retiring server entity instances as capacity changes, or
    B. Introducing a separate capacity abstraction checked by C-Event
conditions.

Recommended default:
  Use option A. Capacity is the available count of server entities for a
server
  entity type. When capacity increases, create additional idle server
entities.
  When capacity decreases, retire only idle excess server entities; busy
servers
  finish naturally, and the engine records a warning if busyCount exceeds
the
  new target capacity.

Extend buildEngine() initialisation after that decision is accepted:
  For each server entity type with a shiftSchedule:
    For each period in shiftSchedule:
      Schedule a SHIFT_CHANGE B-Event at period.time
      Payload: { serverTypeName, newCapacity: period.capacity }

Add SHIFT_CHANGE B-Event handler to phases.js:
  When fired:
    1. Apply the new target capacity for the server entity type using the
       accepted design above
    2. If new capacity < current busyCount, add a warning to the results
object
       and allow busy servers to complete naturally
    3. Write a StepLog entry: "SHIFT_CHANGE: <serverTypeName> capacity ->
<newCapacity>"

Write unit tests:
  - Single server, shift at T=500: capacity 3 → 1
  - Assert: available server capacity behaves as 3 before T=500
  - Assert: available server capacity behaves as 1 after T=500
  - Assert: StepLog contains SHIFT_CHANGE entry at T=500
  - Capacity-below-busyCount edge case: warning in results, run continues
```

Completion checklist:

- [x] ~~SHIFT_CHANGE B-Events scheduled at `buildEngine()` init~~
- [x] ~~SHIFT_CHANGE handler applies server type capacity using the accepted architecture~~
- [x] ~~StepLog entry written on each capacity change~~
- [x] ~~Warning (not halt) if new capacity < current busyCount~~
- [x] ~~Unit tests pass~~

F7.5 — Time-Varying Distribution Picker UI

Audit status: ✗ (DistPicker does not support piecewise type)
Action: Extend `src/ui/shared/components.jsx` (DistPicker)
Status: ⬜ | **Completed:** —

```
Task F7.5 of Sprint 7.

Read src/ui/shared/components.jsx — find the DistPicker component.
Confirm the list of distribution types currently shown.

Extend DistPicker (do not replace) to support the piecewise type:
  - Add "Time-varying (piecewise)" to the distribution type dropdown
  - On select: show a period table editor:
      Columns: Start Time | Distribution Type | Parameters | [Delete]
      Add Period button: appends a new row with defaults
      Period 0 start time is always locked to 0 (cannot be changed by user)
      Periods are auto-sorted ascending by startTime on save
  - Each period row uses a nested DistPicker (excluding piecewise — no
recursion)
  - Validates inline: unsorted times highlighted in red; warns if last
period
    ends before run duration

Write a UI test (add to tests/ui/shared/dist-picker.test.jsx):
  - Selecting piecewise shows period table
  - Adding a period inserts a new row
  - Period 0 start time field is disabled
  - Unsorted start times show validation error
```

Completion checklist:

- [x] ~~Piecewise type in existing `DistPicker` dropdown~~
- [x] ~~Period table editor: add, edit, delete rows~~
- [x] ~~Period 0 start time locked to 0~~
- [x] ~~Nested `DistPicker` per period (no recursion)~~
- [x] ~~Inline sort validation~~
- [x] ~~UI test passes~~

F7.6 — Shift Schedule Editor

Audit status: ✗ (server entity type configuration has no shift schedule panel)
Action: Extend server entity type configuration in `editors/index.jsx`
Status: ⬜ | **Completed:** —

```
Task F7.6 of Sprint 7.

Read src/ui/editors/index.jsx — find the entity type configuration section.
Show me the current fields rendered for a server entity type (`role:
"server"`).

Extend the server entity type configuration panel (do not replace):
  Add a "Shift Schedule" section (collapsed by default):
    - Toggle: "Use shift schedule" (checkbox)
    - When enabled: show a schedule table
        Columns: Start Time | Capacity | [Delete]
        Add Shift button: appends a new row
        Row 0 time locked to 0 (defines initial capacity)
    - When disabled: show the existing static Capacity field only
    - Inline validation: times must be sorted; capacity must be a positive
integer

  On save: if shift schedule enabled, write shiftSchedule[] to model_json.
           Static count/capacity field is ignored when shiftSchedule[0] is
present.

Write a UI test:
  - Enabling shift schedule shows the table; disabling restores static
capacity field
  - Adding a row increments table row count
  - Non-integer capacity value shows validation error
```

Completion checklist:

- ~~Shift schedule section in existing server entity type editor~~
- ~~Toggle enables/disables shift table vs static capacity field~~
- ~~Row 0 time locked to 0~~
- ~~Inline validation: sorted times, positive integer capacity~~
- ~~`shiftSchedule[]` written to model_json when enabled~~
- ~~UI test passes~~

F7.7 — Validation for Time-Varying Parameters

Audit status: ✗ (validateModel() does not cover piecewise or shiftSchedule)
Action: Extend `src/engine/validation.js`
Status: ⬜ | **Completed:** —

```
Task F7.7 of Sprint 7.
```

```
Read src/engine/validation.js — show me the full validateModel() function
and the existing V1–V11 rule set.

Add validation rules V12–V15 to the existing validateModel():

  V12 — Piecewise distribution: periods[0].startTime must be 0. (Blocking)
  V13 — Piecewise distribution: periods must be sorted ascending by
startTime. (Blocking)
  V14 — Shift schedule: shiftSchedule[0].time must be 0. (Blocking)
  V15 — Shift schedule: all shift times must be within run duration if run
        duration is configured. (Warning — run proceeds with amber banner)

Error message patterns:
  V12: "Arrival event '{name}' piecewise distribution must start at time 0."
  V13: "Arrival event '{name}' piecewise periods are not sorted by start
time."
  V14: "Server type '{name}' shift schedule must begin at time 0."
  V15: "Server type '{name}' has shift times beyond run duration — later
shifts will not fire."

Add to tests/engine/validation.test.js (extend existing file):
  - V12: model with piecewise period[0].startTime != 0 returns blocking
error
  - V13: unsorted piecewise periods return blocking error
  - V14: shift schedule without time=0 period returns blocking error
  - V15: shift time beyond run duration returns warning (not blocking)
  - Valid piecewise + shift model passes validation
```

Completion checklist:

- ~~V12–V15 added to existing `validateModel()`~~
- ~~V12, V13, V14 are blocking — run prevented~~
- ~~V15 is a warning — run proceeds with banner~~
- ~~Error messages identify the specific node by name~~
- ~~Five new validation tests pass~~
- ~~Existing V1–V11 tests unaffected~~

Sprint 7 Completion Gate

```
npm test -- --run                    # Zero failures
npm test -- distributions            # Piecewise distribution tests pass
npm test -- engine                   # NHPP + shift change tests pass
npm test -- validation               # V12–V15 tests pass
npm test -- ui                       # Shift editor + DistPicker UI tests
pass
```

```
node tests/[benchmark-filename]        # M/M/1 benchmark still exits 0
npm run build                          # Succeeds
```

Sprint 8A — LLM Provider Architecture Preflight

Goal: Prepare the LLM infrastructure for AI Generated Model Authoring without binding Sprint 8 more deeply to a single provider. This sprint keeps all provider keys server-side and introduces a provider-neutral request/response boundary.

Status: ✅ Complete | **Started:** 2026-05-05 | **Completed:** 2026-05-05
Prerequisite: Sprint 7B complete. Complete this before Sprint 8 implementation.

Feature	Audit Status	Action
F8A.1 — Provider-neutral LLM request contract	✅	Added `src/llm/contracts.js` with task, response format, and neutral request shape
F8A.2 — Edge Function provider router	✅	Refactored `llm-proxy` around server-side provider routing with Anthropic as the first implementation
F8A.3 — Server-side model/provider configuration	✅	Added `LLM_PROVIDER`, `LLM_MODEL`, and `ANTHROPIC_MODEL` server-side config path; no browser API keys
F8A.4 — Browser API compatibility	✅	Preserved `streamNarrative()` API while sending neutral requests through the proxy
F8A.5 — Tests and deployment checklist	✅	Added contract/API/proxy tests and deployment notes

Design Principles for Sprint 8A

- Do not expose provider API keys in browser code.
- Do not build a full admin LLM settings UI unless explicitly scoped.
- Preserve existing Sprint 6 AI results analysis behavior.
- Keep model-builder prompts in Sprint 8 provider-neutral from the start.

Sprint 8A Completion Gate

```
npm test -- llm execute-panel
npm run build
# Manual: deploy llm-proxy and verify one AI Insights request still streams
```

Sprint 8A Completion Notes

- Browser code now sends provider-neutral LLM requests with `version`, `kind`, `messages`, `maxTokens`, `stream`, and `responseFormat`.
- Browser code no longer sends a provider or model choice in normal `streamNarrative()` calls.
- `llm-proxy` selects provider/model server-side using `LLM_PROVIDER`, `LLM_MODEL`, or `ANTHROPIC_MODEL`, with the existing Anthropic secret remaining server-only.
- The Edge Function still accepts the legacy Sprint 6 payload shape for deployment compatibility during rollout.
- Verification passed: `npm test -- llm execute-panel` (5 files, 22 tests) and `npm run typecheck`.
- Deployment still requires redeploying `llm-proxy` and verifying one live AI Insights stream.

Sprint 8 — AI Generated Model Authoring

Goal: Add the second model authoring mode from ADR-007. A modeller describes a system in natural language and receives a partially- or fully-configured DES model proposal. The LLM acts as a model-construction assistant: it parses the description, asks clarifying questions, proposes entity classes, queues, B-Events, C-Events, and distributions, and presents the result as canonical `model_json`. The modeller reviews a diff-preview before any changes are applied.

Status: ✅ Complete | **Started:** 2026-05-05 | **Completed:** 2026-05-05
Prerequisite: Sprint 8A complete. Sprint 8 depends on the Supabase Edge Function `llm-proxy`, but model-builder calls must use the provider-neutral contract introduced in Sprint 8A.

Architectural constraint: ADR-007 applies. AI Generated Model is an authoring mode over the same canonical `model_json` used by Forms/Tabs and the future Visual Designer. The LLM produces a model JSON object conforming to the existing `addition1_entity_model.md` schema. It is validated by `validateModel()` before being applied. The engine, macros, and database schema are not modified. The LLM cannot bypass validation.

Feature	Audit Status	Action
F8.1 — AI Generated Model panel	✅	Added first tab in model editor with conversation UI and proposal panel
F8.2 — Intent parser prompt + LLM schema	✅	Added `src/llm/model-builder-prompts.js`

Feature	Audit Status	Action
F8.3 — Iterative clarification loop	✅	Added non-streaming model-builder API call and multi-turn panel handling
F8.4 — Diff-preview before apply	✅	Added structured `ModelDiffPreview`
F8.5 — Validation integration	✅	Proposal and merged partial apply paths call `validateModel()` before applying
F8.6 — Partial model apply	✅	Added section-level partial apply

Design Principles for Sprint 8

Sprint 8 implements the AI Generated Model authoring mode. It does not replace the Forms/Tabs editor and does not introduce a separate model format.

The LLM is constrained to produce only valid model JSON. The system prompt is extensive and encodes the full entity model schema (addition1_entity_model.md Section 2–7) as constraints on LLM output. The LLM must not invent macros, distribution types, or field names outside the documented schema.

The builder operates in three modes:

Describe → Build: User writes a free-text description of the system (e.g., "An emergency department with triage, nurse assessment, and doctor consultation. Patients arrive at exponential rate 0.3, triage takes 5–10 minutes uniformly..."). The LLM returns a complete model JSON proposal.

Refine: User types a refinement request against the current model (e.g., "Add a second doctor and a priority queue for critical patients"). The LLM returns the full proposed model after the refinement. The UI computes the structured diff locally; any LLM-written diff narrative is advisory only and must not be treated as authoritative.

Explain: User asks "What does this C-Event do?" — the LLM returns a plain-English explanation of the selected model element.

All three modes route through the same `llm-proxy` Edge Function. The mode is encoded in the system prompt. The API key and provider selection remain server-side in the Edge Function.

Sprint 8 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 8 of DES Studio.

Sprint 8 adds the AI Generated Model authoring mode: an LLM-powered
```

```
chat interface for building simulation models from natural language
descriptions.

Completed in Sprints 1–6: [paste — note: Sprint 6 must be complete]

Features:
  F8.1 — AI Generated Model panel: new panel in model editor view
  F8.2 — Intent parser prompt: system prompt encoding full entity model
schema
  F8.3 — Iterative clarification loop: multi-turn conversation state
  F8.4 — Diff-preview: structured diff between current and proposed model
  F8.5 — Validation integration: validateModel() on proposed model before
apply
  F8.6 — Partial apply: user selects which sections of proposed model to
accept

CRITICAL:
  - LLM output is ALWAYS validated before being applied — no exceptions
  - LLM cannot produce macros, fields, or distribution types outside the
schema
  - The existing model (if any) is included in the LLM context — the builder
    refines, not replaces, unless the user explicitly requests a full
rebuild
  - Conversation history is stored in React state only — not persisted to
Supabase
  - The builder panel is additive — existing tab editors remain fully
functional
  - ADR-007 applies — this is the second authoring mode over canonical
model_json
  - Maximum conversation turns before auto-summary: 10 (prevents context
overflow)

Definition of done:
  - User types "a post office with 2 clerks, FIFO queue, exponential
arrivals at rate 0.5"
  - LLM proposes model JSON, user sees diff-preview, clicks Apply
  - Model loads into existing editors correctly and passes validateModel()
  - User types "add a priority queue for express customers"
  - LLM proposes diff, user applies only that section
  - npm test -- --run passes
```

F8.1 — AI Generated Model Panel

Audit status: ✗ (model editor has no chat interface)
Action: New panel alongside existing `editors/index.jsx` tab structure
Status: ✅ | **Completed:** 2026-05-05

```
Task F8.1 of Sprint 8.

Read src/ui/editors/index.jsx — understand the existing tab structure.

Add a new "AI Generated Model" tab to the existing model editor tab bar:
  - Position: first tab (leftmost) — it is the entry point for new models
  - Tab label: "AI Generated Model"
  - Panel structure:
      Top: conversation history (scrollable, message bubbles)
      Bottom: text input + Send button
      Right: model proposal panel (hidden until first LLM response)
  - Message bubble types: user (right-aligned), assistant (left-aligned),
    system notices (centred, muted — e.g., "Applying proposal...")
  - Conversation history is React state — not persisted

The tab must not affect any existing tab (entity types, state vars, etc.).
Write a UI test: "AI Generated Model" tab renders; other tabs unaffected.
```

Completion checklist:

- ✅ ~~"AI Generated Model" tab added as first tab in existing editor~~
- ✅ ~~Existing tabs unaffected~~
- ✅ ~~Conversation history renders as message bubbles~~
- ✅ ~~Text input + Send button present~~
- ✅ ~~UI test passes~~

F8.2 — Intent Parser Prompt & LLM Schema

Audit status: ✗ (not implemented)
Action: New `src/llm/model-builder-prompts.js`
Status: ✅ | **Completed:** 2026-05-05

```
Task F8.2 of Sprint 8.

Create src/llm/model-builder-prompts.js (new file).

Export buildModelBuilderSystemPrompt():
  Returns a system prompt that:
    1. Identifies the LLM as a DES Studio model construction assistant
    2. Encodes the complete entity model schema as constraints:
         - Entity classes: name (string), attributes (name, valueType,
defaultValue, mutable)
         - B-Events: name, scheduleRows (macro: ARRIVE, parameters)
         - C-Events: priority (1..N), condition (predicate JSON), actions
(macro rows)
```

```
          - Queues: name, discipline (FIFO|LIFO|PRIORITY)
          - State variables: name, valueType: number, initialValue,
resetOnWarmup
          - Macros: ARRIVE, ASSIGN, COMPLETE, RELEASE, RENEGE (exact
signatures)
          - Distributions: exponential, uniform, normal, triangular, fixed,
lognormal, empirical
    3. Instructs the LLM to respond ONLY in JSON:
          {
            "intent": "build" | "refine" | "clarify",
            "questions": ["..."] | null,  // if intent is clarify
            "proposedModel": { ...model_json... } | null,  // complete
proposed model if intent is build/refine
            "explanation": "..."  // always present — plain-English summary
          }
    4. Instructs the LLM to ask at most 2 clarifying questions before
proposing a model
    5. Forbids the LLM from inventing fields, macros, or distribution types
       outside the documented schema
    6. For refine requests, requires proposedModel to be the complete model
after
       the requested refinement; the UI computes the diff locally

Export buildModelBuilderUserMessage(description, currentModel,
conversationHistory):
  Returns the user message for the current turn, including:
    - Current model JSON (if any — the LLM refines, not replaces)
    - Conversation history (last 10 turns)
    - Current user description or refinement request

Unit tests (tests/llm/model-builder-prompts.test.js):
  - System prompt contains all five macro names
  - System prompt contains all seven distribution types
  - buildModelBuilderUserMessage includes currentModel in output when model
is non-empty
  - Response JSON schema: intent + proposedModel + explanation keys present
```

Completion checklist:

- ~~`buildModelBuilderSystemPrompt()` exported~~
- ~~`buildModelBuilderUserMessage()` exported~~
- ~~System prompt encodes complete schema (macros, distributions, field types)~~
- ~~Response format constrained to intent/questions/proposedModel/explanation JSON~~
- ~~Unit tests pass~~

F8.3 — Iterative Clarification Loop

Audit status: ✗ (not implemented)
Action: Extend chat panel (F8.1); extend `src/llm/apiClient.js`
Status: ✅ | **Completed:** 2026-05-05

```
Task F8.3 of Sprint 8.

Read src/llm/apiClient.js (from F6.2).

Extend apiClient.js to support multi-turn model builder calls:
  Export callModelBuilder(systemPrompt, messages, onComplete, onError):
    - Non-streaming (model builder requires full JSON, not streaming tokens)
    - POST to llm-proxy Edge Function with full conversation history
    - On success: parse response JSON (intent + proposedModel + explanation)
    - On error: call onError with user-friendly message

In the chat panel (F8.1):
  State: conversationHistory (array of {role, content} turns)
  On Send:
    1. Append user message to conversationHistory
    2. Call callModelBuilder with full history
    3. Parse response:
       - If intent == "clarify": display assistant bubble with questions;
         wait for user response (next Send)
       - If intent == "build" or "refine": display explanation bubble;
         show model proposal panel (F8.4)
    4. Append assistant response to conversationHistory
    5. If conversationHistory.length >= 20 turns: show notice
       "Conversation is long — consider starting a new session"

Cap: after 10 assistant turns without a proposal, auto-inject a system
message:
  "Please now produce a model proposal based on the discussion so far."
```

Completion checklist:

- ~~`callModelBuilder()` exported from `apiClient.js`~~
- ~~Multi-turn conversation state managed in React state~~
- ~~Clarification questions render as assistant bubbles~~
- ~~Proposal displayed when intent is build/refine~~
- ~~20-turn length warning shown~~
- ~~10-turn auto-proposal injection~~

F8.4 — Diff-Preview Before Apply

Audit status: ✗ (not implemented)
Action: New `src/ui/editors/ModelDiffPreview.jsx`
Status: ✅ | **Completed:** 2026-05-05

```
Task F8.4 of Sprint 8.

When the LLM returns a proposedModel (intent: build or refine), display
a structured diff view before the user can apply it.

Create src/ui/editors/ModelDiffPreview.jsx (new file):
  Receives: currentModel (model_json), proposedModel (model_json)
  Renders a diff panel with sections:
    Entity Classes: [Added] [Modified] [Removed] [Unchanged]
    B-Events: [Added] [Modified] [Removed] [Unchanged]
    C-Events: [Added] [Modified] [Removed] [Unchanged]
    Queues: [Added] [Modified] [Removed] [Unchanged]
    State Variables: [Added] [Modified] [Removed] [Unchanged]

  Each changed element shows the old and new value side by side.
  Unchanged elements are collapsed (expandable on click).

  Buttons:
    "Apply All" — replaces currentModel with proposedModel in app state
    "Apply Selected" — opens section-level checkboxes; applies only checked
sections
    "Discard" — closes the proposal panel without applying

  Applying triggers validateModel(proposedModel):
    If blocking errors: show error list; do not apply; keep diff panel open
    If warnings only: apply with amber banner listing warnings

Write a unit test: ModelDiffPreview correctly identifies one added entity
class
and one removed queue in a known diff.
```

Completion checklist:

- [x] ~~`ModelDiffPreview.jsx` created~~
- [x] ~~Diff correctly identifies added/modified/removed/unchanged elements~~
- [x] ~~Apply All replaces model in app state~~
- [x] ~~Apply Selected applies only checked sections~~
- [x] ~~Discard closes panel without changes~~
- [x] ~~`validateModel()` called before apply — blocking errors prevent apply~~
- [x] ~~Unit test passes~~

F8.5 — Validation Integration

Audit status: ~ (validateModel() exists — must be called on proposed model)
Action: Reuse existing `validation.js` — no new code
Status: ✅ | **Completed:** 2026-05-05

```
Task F8.5 of Sprint 8.

This task has no new implementation — it is a verification task.

Confirm that ModelDiffPreview (F8.4) correctly calls validateModel()
on the proposedModel before applying.

Also confirm:
  - Blocking errors (V1–V14) prevent apply and show error list in diff panel
  - Warning (V11, V15) allow apply with amber banner
  - Error messages identify specific nodes by name (existing behaviour)

Write an integration test:
  - Proposed model with a missing entity class name (V1 violation)
  - Assert: Apply All is disabled; error list shown; diff panel stays open
  - Proposed model with valid content
  - Assert: Apply All proceeds; model updated in app state
```

Completion checklist:

- [x] ~~`validateModel()` called on proposed model before apply~~
- [x] ~~Blocking errors disable Apply; show list~~
- [x] ~~Warnings allow Apply with banner~~
- [x] ~~Integration test passes~~

F8.6 — Partial Model Apply

Audit status: ✗ (not implemented)
Action: Extend `ModelDiffPreview.jsx`
Status: ✅ | **Completed:** 2026-05-05

```
Task F8.6 of Sprint 8.

Extend ModelDiffPreview (F8.4) to support section-level partial apply:

When user clicks "Apply Selected":
  Show checkboxes at section level (Entity Classes, B-Events, etc.)
  Each section checkbox controls all elements in that section
  "Apply Selected" button applies only checked sections
```

```
Merge logic:
  For each checked section: replace that section in currentModel with
  the corresponding section from proposedModel.
  Unchecked sections retain their currentModel values.

After merge: run validateModel() on the merged model.
  If blocking errors: show errors; do not apply.
  If valid: apply and close diff panel.

Unit test:
  - Apply only "Queues" section from proposedModel
  - Assert: entity classes unchanged; queues updated; no other sections
changed
```

Completion checklist:

- [x] ~~Section-level checkboxes in `ModelDiffPreview`~~
- [x] ~~Apply Selected merges only checked sections~~
- [x] ~~`validateModel()` called on merged model~~
- [x] ~~Unit test passes~~

Sprint 8 Completion Gate

```
npm test -- --run                          # Zero failures
npm test -- llm                            # Model builder prompt tests pass
npm test -- ui                             # Chat panel + diff preview UI tests
pass
npm run build                              # Succeeds
# Manual: type "a bank with 3 tellers, FIFO queue, arrivals at rate 0.4"
# Verify: LLM proposes model; diff-preview shows new elements; Apply creates
valid model
# Manual: "add a second queue for priority customers"
# Verify: LLM proposes diff; partial apply works; validate passes
```

Sprint 8 Implementation Notes

- Local implementation and automated verification are complete for F8.1-F8.6.
- Full suite passed: `npm test` -> 42 files, 385 tests.
- Production build succeeded.
- Refinement: proposal panel includes direct `Apply & Save All` and `Apply & Save Selected` actions because the top-bar Save may not be visible while reviewing AI proposals.

- Manual verification exposed model-definition coherence blockers. Sprint 8B supersedes further Sprint 8 expansion before visual designer work.

Sprint 8B — Model Definition Coherence

Goal: Make the canonical DES model understandable and executable across Forms/Tabs, AI Generated Model, validation, and the engine before adding more authoring surfaces.

Status: ✅ Complete | **Started:** 2026-05-05 | **Completed:** 2026-05-05
Prerequisite: Sprint 8 implementation pass. Complete before Sprint 9A/Sprint 9.

Why This Sprint Blocks Further Work

Manual review in `docs/UI - Observations.md` showed that the current surface is too dependent on macro strings. Users see `ARRIVE(...)`, `ASSIGN(...)`, `COMPLETE()`, and "template" implementation language instead of clear modelling actions. AI generation also exposed gaps: missing effects, false V8 warnings, queue names with spaces, and unclear multi-stage routing.

Proceeding to Visual Designer before resolving this would bake ambiguous semantics into a second authoring surface.

Sprint 8B Feature Set

Feature	Audit Status	Action
F8B.1 — Validation parity for generated/manual effects	✅	`validateModel()` recognises string, array, and object effects; no false V8 for valid ARRIVE/COMPLETE
F8B.2 — Queue names with spaces	✅	Engine macros, condition evaluator, validation, and generated dropdown values support queue names like `Triage Queue`
F8B.3 — Queue/customer binding in dropdowns	✅	Queue `customerType` filters ARRIVE options; only valid customer-to-queue combinations appear
F8B.4 — User-facing action labels	✅	Dropdowns say what the action does, e.g. "Add Patient to Triage Queue", while storing canonical macro values
F8B.5 — Service-start/service-complete pattern	✅	AI normalization generates `condition: queue > 0 AND idle server > 0`, service-start assignment, and follow-on completion scheduling
F8B.6 — Remove implementation language	✅	User-facing follow-on surfaces avoid "template" wording and use scheduled follow-on/plain event

Feature	Audit Status	Action
		names
F8B.7 — Two-stage reference model	✅	Patient -> Queue 1 -> Service 1 -> Queue 2 -> Service 2 -> Complete is covered by focused regression tests
F8B.8 — Save visibility	✅	Dirty state appears in the editor body with a nearby Save Changes action, while the top-bar Save remains available
F8B.9 — AI refinement friendliness	✅	Modified proposal items now show friendly field-level summaries instead of raw JSON blocks

Canonical Two-Stage Acceptance Model

The sprint must prove this shape works manually and through AI normalization:

```
Patient
  -> Triage Queue
  -> Triage Nurse service
  -> Consultant Queue
  -> Consultant service
  -> Complete
```

Expected model elements:

- Entity types: `Patient`, `Triage Nurse`, `Consultant`
- Queues: `Triage Queue` accepts `Patient`; `Consultant Queue` accepts `Patient`
- B-events:
  - `Patient Arrival`: add Patient to Triage Queue and schedule next arrival
  - `Triage Complete`: release Triage Nurse and move Patient to Consultant Queue
  - `Consultation Complete`: release Consultant and mark Patient complete
- C-events:
  - `Start Triage`: condition `queue(Triage Queue).length > 0 AND idle(Triage Nurse).count > 0`; schedules `Triage Complete`
  - `Start Consultation`: condition `queue(Consultant Queue).length > 0 AND idle(Consultant).count > 0`; schedules `Consultation Complete`

Sprint 8B Completion Gate

```
npm test -- validation conditions queue-name-spaces ai-generated-model-panel
b-event-editor c-event-editor model-builder-prompts accessibility
npm run build
```

```
# Manual: create/generated two-stage model above, run it, confirm no V8/V9
false positives and no user-visible "template" wording.
```

Sprint 9A — Visual Designer Architecture Preflight

Goal: Make the dependency and persistence decisions needed for the Visual Designer before building the graph-first authoring surface.

Status: 🔄 In progress | **Started:** 2026-05-05 | **Completed:** —
Prerequisite: Sprint 8B complete. ADR-010 resolves the former `TBD-VISUAL-CANVAS` decision.

Feature	Audit Status	Action
F9A.1 — Canvas dependency decision	✅	Accepted ADR-010: use `@xyflow/react`; do not use old `reactflow` package name
F9A.2 — Graph metadata persistence	✅	Accepted ADR-010: persist optional `model_json.graph` layout metadata only; derive topology
F9A.3 — Round-trip contract	✅	Accepted ADR-010: Forms/Tabs, AI, and Visual Designer edit one canonical model; graph can regenerate
F9A.4 — Inspector reuse strategy	✅	Accepted ADR-010: reuse small building blocks (`DistPicker`, `ConditionBuilder`, `EntityFilterBuilder`, option helpers), not full tab editors

Sprint 9A Completion Gate

```
git diff --check
# Manual: confirm ADR-007, ADR-010, and visual design doc agree with
dependency/metadata decision
```

Sprint 9 — Visual Designer Authoring

Goal: Add the third model authoring mode from ADR-007: a graph-first visual designer for building and editing DES models over the same canonical `model_json` used by Forms/Tabs and AI Generated Model authoring.

Status: 🔄 In progress | **Started:** 2026-05-05 | **Completed:** —
Prerequisite: Sprint 9A complete. The AI Generated Model path should already prove that canonical model proposals can be validated and applied safely. ADR-010 accepts `@xyflow/react` as the canvas dependency and defines `model_json.graph` as optional layout metadata only.

Architectural constraint: Do not implement the retired split-pane SVG hybrid designer. Sprint 9 should target the final graph-first authoring surface directly. The visual designer must not create a separate model format and must not bypass `validateModel()`.

Feature	Audit Status	Action
F9.1 — Visual Designer shell and mode entry	✓	New Visual Designer tab renders derived graph summary and editable canvas
F9.2 — Graph derivation and layout metadata	✓	Derivation helper and layout persistence are implemented; derived edges are not persisted
F9.3 — `@xyflow/react` canvas renderer and node palette	~	Canvas, button palette, and drag-to-place palette creation are implemented; richer palette UX remains
F9.4 — Connection rules and DAG validation	~	Initial conservative connection rules and visual validation summary implemented; richer validation panel deferred
F9.5 — Inspector integration	~	Compact inspector includes timing distribution controls; advanced resource editing and delete workflows deferred
F9.6 — Round-trip parity with Forms/Tabs	~	Visual edits update canonical `model_json`; broader manual browser pass remains

Sprint 9 Current Implementation Contract

The first Sprint 9 deliverable is a reviewable and testable model-authoring surface, not final UX polish. Visual actions update canonical `model_json` first, then the canvas re-derives topology. `model_json.graph` stores layout metadata only.

Deferred UX refinement:

- richer palette placement affordances
- polished validation side panel
- advanced resource editing from the canvas inspector
- delete workflow and richer connection editing

Design Principles for Sprint 9

The Visual Designer is a first-class authoring mode, not a preview of the tab editor and not a temporary SVG bridge. It should let users think in terms of sources, queues, activities, sinks, and connections, while still producing the same model JSON that the engine already runs.

The useful parts of `docs/DES_Studio_Visual_Designer_Design.md` remain valid:

- the four node types
- port rules
- DAG validation rules
- graph layout metadata questions
- inspector reuse as a product concept

The obsolete part is the phased SVG hybrid implementation path. ADR-007 supersedes that path.

Sprint 9 Planning Prompt *(run in claude.ai)*

```
We are starting Sprint 9 of DES Studio.

Sprint 9 adds the Visual Designer authoring mode defined by ADR-007.
The retired split-pane SVG hybrid designer must NOT be implemented.

Completed in Sprints 1–8: [paste — note: Sprint 8 must be complete]

Read:
  - docs/decisions/ADR-007-three-authoring-modes.md
  - docs/DES_Studio_Visual_Designer_Design.md
  - src/ui/editors/index.jsx
  - src/ui/ModelDetail.jsx
  - docs/addition1_entity_model.md

Features:
  F9.1 — Visual Designer mode entry in the model authoring surface
  F9.2 — Decide graph layout metadata strategy for model_json
  F9.3 — Canvas renderer with Source, Queue, Activity, Sink nodes
  F9.4 — Connection rules and DAG validation
  F9.5 — Inspector integration using existing editor components where
practical
  F9.6 — Round-trip parity with Forms/Tabs and AI Generated Model outputs

CRITICAL:
  - One canonical model_json; no visual-designer-only model format
  - Do not build FlowDiagramSVG as a bridge implementation
  - Canvas dependency is accepted in ADR-010: use @xyflow/react, not
reactflow
  - model_json.graph is optional layout metadata only; graph topology is
derived
  - validateModel() remains the gate before execution or save
```

```
  - Forms/Tabs and AI Generated Model remain first-class authoring modes

Definition of done:
  - User can enter Visual Designer authoring mode for an existing model
  - User can create/edit nodes and valid connections
  - Invalid connections and cycles are blocked
  - Changes are reflected in canonical model_json and visible in Forms/Tabs
  - npm test -- --run passes
  - npm run build succeeds
```

Sprint 9 Completion Gate

```
npm test -- --run                          # Zero failures
npm test -- ui                             # Visual designer mode and inspector
tests pass
npm test -- validation                     # DAG/connection validation tests
pass
npm run build                              # Succeeds
# Manual: create a model visually, switch to Forms/Tabs, confirm same data
# Manual: edit a model in Forms/Tabs, switch to Visual Designer, confirm
graph reflects it
# Manual: attempt invalid connections/cycles, confirm they are blocked
```

Completion note: Sprint 9 is complete as the initial reviewable Visual Designer authoring model. The user reviewed the initial model and called out round-tripping as good. Remaining visual-designer UX polish is consolidated into Sprint 9B.

Sprint 9B — Visual Designer UX Hardening

Goal: Harden and polish the Visual Designer now that the core graph-first authoring model and Forms/Tabs round-trip are working.

Status: 🔄 In progress | **Started:** 2026-05-06 | **Completed:** —
Prerequisite: Sprint 9 complete. Continue using the same canonical `model_json`; do not introduce a second graph model.

Architectural constraint: Sprint 9B is refinement over the proven Sprint 9 model. Visual edits still update canonical model data first; `model_json.graph` remains layout metadata only.

Feature	Audit Status	Action
F9B.1 — Activity resource/server	✗	Add server/resource picker in the Activity inspector and update condition/effect canonical strings safely

Feature	Audit Status	Action
editing		
F9B.2 — Safe node deletion workflow	✗	Delete visual nodes through canonical model elements with dependency warnings and re-derived graph
F9B.3 — Connection editing polish	✗	Add edge delete/re-route workflow and clearer connection feedback
F9B.4 — Richer validation panel	~	Expand compact visual validation into a node-linked checklist with canvas highlighting
F9B.5 — Palette and placement affordances	~	Improve drag/drop affordance, drop hints, fit/reset layout, and selected-node clarity
F9B.6 — Browser review and round-trip hardening	~	Manually verify create/connect/edit/save/reload/execute and add regression tests for gaps
F9B.7 — Bundle/code-splitting review	🟪	Consider lazy-loading Visual Designer to reduce main bundle size after React Flow integration
F9B.8 — Review Observations 0605 coherence pass	✅	Address manual-review findings: AI-generated ARRIVE/service defaults, follow-on entity context defaults, clearer import/export/AI labels, B/C-event wording, model run-count refresh, C-Event `> 0` defaults, and clock decimal formatting.
F9B.9 — Entity Attribute Logic Fixes	✅	Restored EntityFilterBuilder to Visual Designer inspector and unified AttrEditor to use DistPicker (allowing non-Fixed distributions and CSV import for attributes)

Sprint 9B Completion Gate

```
npm test -- visual-designer c-event-editor accessibility model-export
npm test -- ai-generated-model-panel b-event-editor c-event-editor model-export model-import accessibility execute-panel
npm run build
# Manual: create, connect, edit resource/timing, save, reload, and execute a visual model
# Manual: delete each node type and confirm dependencies are explained before mutation
# Manual: invalid connections and validation warnings highlight the affected node
# Manual: generate a simple queue with AI and confirm ARRIVE, ASSIGN, COMPLETE scheduling, and Pass Entity Context are populated
```

Deferred Features Register

Feature	Original sprint	Deferred to	Reason	Status
Split-pane SVG hybrid visual designer	Visual designer v2.0 Phase 2	Not scheduled	ADR-007 retired the temporary SVG bridge in favour of direct final Visual Designer authoring	Cancelled
React Flow / canvas dependency decision	Visual designer planning	Sprint 9A	ADR-010 accepts `@xyflow/react` and a narrow vendor-CSS exception	Complete
SaaS tenancy/workspace model	Platform architecture planning	Post-7A architecture sprint	Sprint 7A intentionally handles roles/settings only; tenancy needs separate schema/RLS planning	Deferred
Provider-neutral LLM configuration	Platform architecture planning	Sprint 8A	Sprint 7A intentionally excludes LLM provider switching; future LLM authoring should not deepen browser/provider coupling	Planned
Admin-selectable LLM provider/model	Platform architecture planning	Sprint 8A or later admin UI sprint	Server-side provider configuration should be prepared before Sprint 8, but a full admin UI can wait	Planned
Architecture health review	Platform architecture planning	Post-7A architecture sprint	Important but broader than the focused roles/settings/TypeScript sprint	Deferred
Execute running-model UX redesign direction	Platform architecture planning	Dedicated UX/design sprint	Execute is MVP-good but needs a separate design pass before major redesign	Deferred
Platform role/settings persistence implementation	Sprint 7A	Sprint 7B	ADR-008 accepted; minimal DB/type foundation implemented before Sprint 7 schema work	Complete

Feature	Original sprint	Deferred to	Reason	Status
TypeScript tooling and first schema contracts	Sprint 7A	Sprint 7B	ADR-009 accepted; tooling/contracts added before schema-heavy model changes	Complete
Apply Sprint 7B Supabase migration to remote project	Sprint 7B	Before settings UI depends on it	Migration file is committed, but remote DB application is a deployment step	Planned
Dependency audit remediation	Sprint 7B	Dependency maintenance sprint	Current fix path requires breaking Vite 8 upgrade; avoid mixing with feature work	Deferred

---

Open Architectural Decisions

ADR	Question	Needed by	Status
ADR-002	Public model run permissions	Sprint 3 F3.8	✅ Accepted — fork model; see `docs/decisions/ADR-002-public-model-run-permissions.md`
ADR-007	Three authoring modes over one canonical model	Sprint 6 planning	✅ Accepted — Forms/Tabs, AI Generated Model, and Visual Designer share canonical `model_json`; SVG hybrid retired
ADR-008	Platform roles and user settings, with SaaS/LLM follow-ons	Sprint 7A / later architecture work	✅ Accepted — platform roles use `profiles.role`; user settings use dedicated table; tenancy/workspaces and LLM provider configuration are deferred follow-on decisions
ADR-009	Reassess JavaScript-only implementation and TypeScript adoption	Sprint 7A	✅ Accepted — adopt TypeScript incrementally at schema/domain boundaries; no broad rewrite
TBD-LLM-API	LLM API key handling — client-side vs Edge Function proxy	Sprint 6 F6.2	✅ Resolved in plan — use Supabase Edge Function proxy; create ADR if implementation changes
TBD-LLM-CONV	LLM conversation persistence —	Sprint 8 F8.3	✅ Resolved in plan — session only React state; create ADR if

ADR	Question	Needed by	Status
	session only vs Supabase		persistence is added
TBD-PIECEWISE-WARMUP	Piecewise distribution warm-up interaction — does warm-up reset the period position?	Sprint 7 F7.2	✅ Resolved in plan — no reset; schedule is time-indexed
TBD-SHIFT-CAPACITY	Shift schedule capacity mapping to current server entity model	Sprint 7 F7.4	✅ Resolved — use server instance scaling; capacity increases create idle instances, decreases retire idle excess, busy excess completes with warning
ADR-010	Visual Designer canvas and graph metadata	Sprint 9A / Sprint 9	✅ Accepted — use `@xyflow/react`; persist optional layout-only `model_json.graph`; derive topology from canonical model logic
(add rows as new questions arise)			

Known Issues Tracker

#	Severity	Finding	File	Sprint	Status
C1	Critical	`new Function()` eval — XSS	`conditions.js:76`, `macros.js:220`	S1 F1.1	✅
C2	Critical	LIFO/Priority ignored by engine	`entities.js:84-88`	S1 F1.3	✅
C3	High	C-scan restart pass-granularity	`engine/index.js:121-142`	S1 F1.2	✅
C4	High	Phase C truncation silent	`engine/index.js:121`	S1 F1.6	✅
C5	High	No pre-run validation	`execute/index.jsx:141-147`	S1 F1.5	✅
C6	Medium	Stale B-Event refs after delete	`phases.js:91,125`	S1 F1.5	✅
C7	Medium	No seeded RNG	`distributions.js:84`	S1 F1.4	✅

#	Severity	Finding	File	Sprint	Status
C8	Low	ConditionBuilder tokens stale	`editors/index.jsx:495`	S2 F2.6	✅
C9	Low	avg_service_time column mismatch	`db/models.js:127`	S5 F5.6	✅
C10	Low	Distribution inputs missing type=number	`editors/index.jsx:171,241,731`	S1 F1.5	✅
C11	Low	DistPicker ReferenceError	`components.jsx`	S1 F1.6	✅

End of build plan. Update after each sprint.
The most important rule: read the existing file before changing it.